STARLET — служебные рутины

Руководство программиста

STARLET-PG-0171-RUS

STARLET Version 1.7.1 *** 31-OCT-2026

Copyright © 2026 StarLet Squad and Ruslan R. Laishev.

StarLet - свободная программа, распространяется на условиях Apache License, Version 2.0. Она поставляется БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ.

Содержание

Предисловие **5**
Кому адресовано . . . 5
Состав руководства . . . 6
Связанные документы . . . 7
Благодарности . . . 8
Товарные знаки . . . 9
Обозначения . . . 10

Глава 1. Замысел **11**
1.1 Зачем нужен StarLet . . . 11
1.2 Что такое StarLet . . . 12
1.3 Чем StarLet не является . . . 13
1.4 Почему этим рутинам стоит существовать . . . 14
1.5 Решения и их причины . . . 16
1.6 Кто его использует . . . 17
1.7 План обучения . . . 18

Глава 2. Начало работы **19**
2.1 Что понадобится . . . 19
2.2 Сборка . . . 20
2.3 Установка в виде пакета . . . 21
2.4 Без CMake . . . 22
2.5 Первая программа . . . 23
2.6 Что сделала программа . . . 24

Глава 3. Условные значения и сообщения **25**
3.1 Устройство условного значения . . . 25
3.2 Серьёзности и младший бит . . . 26
3.3 Коды, которые возвращает библиотека . . . 27
3.4 Определение средства и его сообщений . . . 28
3.5 Каталог . . . 29
3.6 Сигнализация сообщения . . . 30
3.7 Получение текста без печати . . . 31

Глава 4. Журнал **32**
4.1 Рутины с первого взгляда . . . 32
4.2 Запись . . . 33
4.3 `$LOG_FAO` . . . 34
4.4 Куда идёт журнал . . . 35
4.5 Ротация . . . 36
4.6 Syslog и Logcat . . . 37
4.7 Трассировка и дамп . . . 38
4.8 Записи в буфер . . . 39

Глава 5. FAO — форматировщик **40**
5.1 Зачем FAO . . . 40

5.2 Параметры . . . 41
5.3 Директивы . . . 42
5.4 Обзор настоящего вывода . . . 44
5.5 FAO в сообщениях и журналах . . . 45
5.6 Подводные камни . . . 46

Глава 6. Строки и данные **47**
6.1 Счётная строка . . . 47
6.2 Перемещение и поиск . . . 49
6.3 Очистка текста . . . 50
6.4 Шаблоны . . . 51
6.5 Base64 . . . 52
6.6 Текст hex и UUID . . . 53
6.7 CRC-32 . . . 54
6.8 Списки элементов . . . 55
6.9 Мелочи . . . 56

Глава 7. Ключевые слова и параметры **57**
7.1 Поиск ключевого слова . . . 57
7.2 От значений к словам . . . 59
7.3 Параметры командной строки и файла . . . 60

Глава 8. Командная строка в стиле DCL **63**
8.1 Замысел . . . 63
8.2 Таблицы . . . 64
8.3 Законченная программа . . . 66
8.4 Рутины . . . 69
8.5 Правила разборщика . . . 71
8.6 Чем это отличается от DCL . . . 72

Глава 9. Очереди и блокировки **73**
9.1 Наследие . . . 73
9.2 Типы . . . 74
9.3 Операции . . . 75
9.4 Атомарные операции . . . 76

Глава 10. Время **77**
10.1 Двоичное время здесь . . . 77
10.2 Текст . . . 78
10.3 Арифметика . . . 79
10.4 Остальные рутины времени . . . 80

Глава 11. Протокол TLV **81**
11.1 Замысел . . . 81
11.2 Рутины . . . 82
11.3 Предостережения . . . 84

Глава 12. Переносимость **85**
12.1 Что где проверено . . . 85
12.2 Правила, которым следует код . . . 86

Глава 13. Манера письма **87**
13.1 Имена . . . 88
13.2 Оформление . . . 89
13.3 Опознавание и правка . . . 90
13.4 История изменений и функциональное описание . . . 91
13.5 Условные значения, а не печать . . . 92

Глава 14. Тесты **93**
14.1 Что есть . . . 93

Приложение A. Краткая справочная карта **96**

Приложение B. История библиотеки **98**

Приложение C. Известные ограничения: что остаётся в силе **102**

Приложение D. Изменения в версиях 1.7.0 и 1.7.1 **104**
Изменения в версии 1.7.1 104
Изменения в версии 1.7.0 106

Предисловие

Кому адресовано

Руководство предназначено программисту, который хочет использовать StarLet в собственной программе, и тому, кому нужно изменять сам StarLet: добавить директиву, сообщение, квалификатор или рутину, исправить дефект или убедиться, что правка безопасна. Предполагается знание Си, оболочки Linux и умение читать CMake. Знание OpenVMS не предполагается. Но тот, кто OpenVMS знает, освоится за несколько минут и обнаружит, что иные названия — старые друзья.

Студент найдёт здесь небольшую библиотеку, которую можно прочесть от первой строки до последней за несколько вечеров и в которой объяснено, почему каждая часть устроена именно так. Ветеран найдёт то, что помнит: младший бит условного значения, счётную строку, управляющую строку `$FA0`, квалификатор с его формой `/NO` — и, надеемся, простит те места, где программе для Linux пришлось поступить иначе.

Справочное руководство STARLET (STARLET Reference Manual) перечисляет все рутины по одной. Это руководство учит пользоваться ими вместе, и одно другого не повторяет.

Состав руководства

Часть	Содержание
Глава 1	Зачем нужен StarLet, что он есть и чем не является, чем хороши его рутины, план обучения
Глава 2	Начало работы: сборка, установка и первая законченная программа
Глава 3	Условные значения и каталог сообщений
Глава 4	Журнал: записи, файлы, ротация, трассировка и шестнадцатеричные дампы
Глава 5	FAO, форматировщик: все директивы с примерами
Глава 6	Строки и данные: счётные строки, шаблоны, base64, hex, CRC-32, списки элементов
Глава 7	Ключевые слова и параметры: таблицы поиска, параметры командной строки, файлы настройки
Глава 8	Командная строка в стиле DCL: глаголы, параметры, квалификаторы, справка
Глава 9	Очереди, спин-блокировки и атомарные операции
Глава 10	Время: двоичное время, дельта-времена, ASCII-формы
Глава 11	Протокол TLV
Глава 12	Переносимость: Linux, ARMv8, Windows, Android, musl
Глава 13	Манера письма исходных текстов
Глава 14	Тесты и то, что каждый из них стережёт
Приложение A	Краткая справочная карта
Приложение B	История библиотеки
Приложение C	Известные ограничения: что остаётся в силе
Приложение D	Изменения в версиях 1.7.0 и 1.7.1

Связанные документы

- *Справочное руководство STARLET* (STARLET Reference Manual, номер издания STARLET-RM-0171): каждая рутина и каждый макрос в той раскладке, какая принята в справочниках библиотеки времени выполнения.
- Сами исходные тексты. Каждый модуль открывается своим опознаванием и историей изменений, а каждая рутина несёт блок `FUNCTIONAL DESCRIPTION`. Если руководство и этот блок расходятся, прав блок, а затем код, и исправлять следует руководство.

Благодарности

Авторы благодарят Claude — ИИ-модель компании Anthropic, — которая прочла исходные тексты, написала и запустила тесты и примеры и прошла вместе с ними через это руководство.

Авторы благодарят соратников из сообщества NaPi Lab, оплативших подписку на Claude, — благодаря им инженер почти пенсионного возраста смог продолжить эту работу.

И авторы благодарят инженеров Digital Equipment Corporation, чьи рабочие привычки — единственная причина, по которой эта библиотека выглядит так, как выглядит. Большинство из них никогда не прочтёт эти строки. Надеемся, они узнали бы результат.

Товарные знаки

OpenVMS, VAX, Alpha, Digital и DEC — товарные знаки Hewlett Packard Enterprise Development LP или VMS Software, Inc. Linux — зарегистрированный товарный знак Линуса Торвальдса в США и других странах. Windows — товарный знак группы компаний Microsoft. Android — товарный знак Google LLC. Arm — товарный знак Arm Limited. Intel — товарный знак Intel Corporation. GLib — часть проекта GNOME. APR — проект Apache Software Foundation. MinGW-w64 и musl — свободные проекты их авторов. Claude — товарный знак Anthropic, PBC. Все прочие названия продуктов и компаний — товарные знаки их владельцев.

StarLet — независимая работа. Он никак не связан с названными здесь владельцами, ими не одобрен и не поддерживается; их названия приводятся только для того, чтобы сказать, на что StarLet похож и где он работает.

Обозначения

Номера строк не приводятся: они устаревают. Вместо этого называется рутина.

Обозначение	Значение
моноширинный	Рутина, макрос, константа, файл или фрагмент кода
`__util$name`	Рутина библиотеки; `$` в имени — обычный символ, как и в OpenVMS
`$NAME`	Макрос библиотеки
`STS$K_...`	Константа: серьёзность или код завершения
`!AZ`, `!UL`	Директива управляющей строки FAO (глава 5)
status	Условное значение, целое число, которое возвращает рутина; по его младшему биту отличают успех от неудачи
`[x]` в синопсисе	Необязательный аргумент

Примеры кода в этом руководстве скомпилированы и запущены на исходных текстах StarLet 1.7.1. Показан тот вывод, который они дали, за исключением даты, времени и номера потока в записи журнала: они типичны и на вашей машине будут другими.

Глава 1. Замысел

1.1 Зачем нужен StarLet

Программы пишут люди, а люди забывают. Функция возвращает -1, а вызывающий туда не смотрит. Сообщение форматируется в буфер, который оказался чуть мал. Строка журнала говорит «ошибка» и ничего больше, и через три года никто не помнит, какая именно. Командную строку разбирает самодельный цикл, который принимает `-v`, но не принимает `-verbose`, а следующая программа того же автора разбирает её по-другому.

Инженеры Digital Equipment Corporation столкнулись с этими бедами рано и ответили на них не одним хитрым инструментом, а привычкой. Каждая рутина возвращает условное значение (condition value), и младший бит говорит, получилось ли. У каждого сообщения есть средство (facility), номер и серьёзность (severity), а его текст живёт в таблице, а не посреди функции. У каждой строки длина всегда при ней. Каждая команда — это глагол (verb) с параметрами и квалификаторами, записанный в таблице, так что разбор делает система, а не настроение автора в тот день. Эта привычка — причина, по которой программы OpenVMS 1985 года до сих пор работают и до сих пор рассказывают, что с ними не так.

StarLet переносит эту привычку на Linux, а оттуда — на платы ARMv8, на Windows и на Android. Это небольшая библиотека на Си11. Она не эмулирует OpenVMS и не притворяется ею. Это *форма* библиотеки времени выполнения OpenVMS, заставленная работать на тех машинах, что стоят на наших столах сегодня, чтобы программу, написанную сегодня, можно было написать так, как писали хорошие программы тогда.

1.2 Что такое StarLet

StarLet — одна статическая библиотека `libstarlet.a`, составленная из четырёх модулей.

Модуль	Префикс	Что даёт
`utility_routines`	`__util$`, `$LOG`	Журнал; каталог сообщений; условные значения; счётные строки; очереди со спин-блокировками; параметры командной строки и файла настройки; поиск по ключевым словам; base64; CRC-32; текст hex и UUID; сопоставление с шаблоном; и другое
`starlet`	`__util$`	Форматировщик FAO, `__util$fao` и `__util$faol`; двоичное время, `__util$bintim`, `__util$asctim`, `__util$add_times`, `__util$sub_times`
`cli_routines`	`__cli$`	Разборщик команд в стиле DCL: глагол, параметры, квалификаторы, типизированные значения, диспетчеризация, справка
`avproto`	`__tlv$`	Компактный двоичный протокол «тег — длина — значение» (TLV)

Названию нужно слово пояснения. В OpenVMS `STARLET` — это библиотека, в которой объявлены системные сервисы: `SYS$FAO`, `SYS$BINTIM`, `SYS$ASCTIM` и их родня. Модуль `starlet` этого пакета содержит переносимые аналоги нескольких из них, и весь пакет получил имя от него.

Библиотеки OpenVMS — родители идей, собранных здесь, и их имена служат *подсказкой, где искать*, когда у рутины как будто нет причины быть такой: `SYS$FAO`, `SYS$PUTMSG`, `SYS$BINTIM`, `SYS$ASCTIM`, `LIB$ADD_TIMES`, `LIB$SUB_TIMES`, `LIB$MOVC5`, `CLI$PRESENT`, `CLI$GET_VALUE`, а также команды `INSQUE` и `REMQUE`. Но каждая рутина StarLet — своя собственная, со своей последовательностью вызова, и главы 5-11 прямо говорят, чем она отличается.

1.3 Чем StarLet не является

- **Он не заменяет библиотеку Си.** Он стоит поверх неё. Память, файлы, сокеты, потоки, часы — всё это по-прежнему дело libc и ядра.
- **Он не каркас (framework).** Здесь нет главного цикла, нет объектной системы, нет механизма подключаемых модулей. Вы вызываете рутину, она возвращает условное значение, и вы идёте дальше.
- **Он не даёт контейнеров.** Есть очередь, потому что она была у VAX. Нет ни хеш-таблицы, ни дерева, ни вектора.
- **Он не порт библиотеки времени выполнения OpenVMS.** В нём нет векторов сообщений, нет обработчиков условий, нет `LIB$GET_INPUT`, нет дескрипторов полного семейства OpenVMS, нет `SYS$QIO`. Когда заимствовано имя из OpenVMS, стоящая за ним рутина делает то, что сказано в этом руководстве, и не более того.
- **Он не занимается кодировками текста.** Строка — это последовательность октетов. Что эти октеты означают — дело программы.

1.4 Почему этим рутинам стоит существовать

Справедливый читатель спросит: библиотека Си, GLib и Apache Portable Runtime уже существуют и превосходны. Зачем ещё одна? Вопрос правильный, и ответ не «StarLet умеет больше». Он умеет меньше. Он держится одной дисциплины от первой рутины до последней, и эта дисциплина и есть продукт. Вот, без прикрас, что у него получилось и где каждый пункт перестаёт быть верным.

1. Сбой не может спрятаться за соглашением, которое никто не записал. Рутины, которые могут потерпеть неудачу, возвращают условное значение, и проверка всегда одна: `1 & status`. У успеха младший бит взведён, у неудачи — нет. Нет таблицы вида «эта возвращает ноль при успехе, та -1, третья указатель или NULL». Студент выучивает одну проверку и может читать любой вызов. Ветеран знает её как `SS$_NORMAL` — нечётное значит «хорошо». *Где это кончается:* не всякая рутина может потерпеть неудачу. `__util$crc32c` возвращает контрольную сумму, `__util$pattern_match` — истину или ложь, помощники hex — длину. И библиотека не может заставить вызывающего посмотреть: она может лишь сделать правильную проверку лёгкой. Движок FAO, например, только возвращает свои жалобы и ничего не пишет в журнал, а `$LOG_FAO` на них не смотрит (раздел 5.6).

2. Один каталог, один вид у каждой диагностики. Сообщение объявляется один раз, в таблице: его средство, номер, серьёзность и текст. Когда оно сигнализируется, вывод всегда имеет вид `%FAC-S-IDENT, text` — вид, который каждый пользователь OpenVMS читал десять тысяч раз. Текст не вклеен в код, поэтому его можно исправлять, рецензировать и переводить в одном месте, а то же условное значение говорит программе, *что именно* пошло не так, и никому не приходится разбирать предложение.

3. Длины путешествуют вместе с данными. Счётная строка (`ASC`, ASCIC в OpenVMS) несёт длину в своём первом октете. Рутины, заполняющие буфер, принимают буфер, его размер и место, куда вернуть длину: `__util$base64`, `__util$log2buf`, `__util$faol`, `__tlv$get`. Никому не надо гадать, где кончается буфер, и никому не надо снова вызывать `strlen` для строки, длина которой была известна строкой раньше. Форматировщик, которому не хватило места, усекает вывод и *сообщает об этом* в возвращаемом значении; он не пишет за конец буфера — даже для управляющей строки, оборванной посередине директивы, и даже для записи TLV, утверждающей, что в ней больше октетов, чем в пакете. *Где это кончается:* `ASC` вмещает не более 255 октетов, а некоторые рутины всё ещё принимают строки с нулём в конце, потому что так устроен Си.

4. Один форматировщик на всё. `$LOG_FAO`, `$PUTMSG_FAO` и `__util$fao` приводятся в движение одним и тем же движком с одними и теми же директивами. Строка, записанная в журнал, сообщение в каталоге и строка, отформатированная для экрана, используют одни и те же `!UL`, `!AZ`, `!XL`. Движок умеет то, чего не умеет `printf`: `!%S` для множественного числа, `!%D` для даты, `!n<...!>` для поля фиксированной ширины и четыре директивы для объектов повседневной жизни сетевого программиста: `!%M` — адрес MAC, `!%I` — адрес IPv4, `!%6` — адрес IPv6 в каноническом виде RFC 5952, `!%U` — UUID. Он не читает стек случайно: макросы сами считают параметры.

5. Запись журнала говорит, кто, когда и где. Каждая запись несёт дату и время, номер потока, средство и серьёзность. В сборке без `NDEBUG` она несёт ещё и модуль, рутину и строку вызова. Запись собирается в буфере и уходит одним `write()`, поэтому два потока не рвут строки друг друга. *Где это кончается:* модуль, рутина и строка — свойство отладочной сборки. Потребитель, собранный как Release, получает более короткую форму.

6. Малая, статическая, без зависимостей. Четыре модуля, одна статическая библиотека, около одиннадцати тысяч строк вместе с комментариями. Библиотеке нужна библиотека Си и ничего больше (и Winsock в Windows). Это простой Си11. Её добавляют в программу тремя строками CMake, и её можно прочесть всю.

7. Она проверена, и не на одной libc. Двадцать четыре тестовые программы запускаются CTest — двадцать пять прогонов, так как CRC-32 запускается дважды, — а задача непрерывной интеграции собирает их с glibc под AddressSanitizer и UndefinedBehaviorSanitizer и ещё раз с musl. Тесты на враждебный ввод (оборванная управляющая строка, TLV, лгущий о своей длине) выделяют буферы точно по размеру, так что один лишний октет смертелен под санитайзером. Сборки для Windows прогоняются под Wine, а ротация журнала — и в Windows 11. Глава 14 говорит, что стережёт каждый тест.

8. Командная строка в манере DCL, на Си. Глаголы и подглаголы, позиционные параметры, квалификаторы с `/name=value` или `--name=value`, отрицательная форма `/NOname`, списки значений `/tag=(a,b,c)`, умолчания, обязательные элементы, типизированные значения (число, файл, время, адрес MAC, UUID, ключевое слово), сокращение имени до однозначной приставки и экран справки, собранный из тех же таблиц. *Где это кончается:* нет компилятора CLD — таблицы являются инициализаторами Си, — значение нужно присоединять через `=` (`/blocks 5` отвергается с сообщением, которое так и говорит), а некоторые углы шероховаты сильнее, чем в DCL. Глава 8 и приложение C честны на этот счёт.

Ничто из этого не подвиг. Это перечень того, что в 1985 году было *очевидно правильным* и что современная программа на Си всё ещё может иметь, если кто-то потрудится собрать это в одном месте. Вот это место.

1.5 Решения и их причины

Младший бит — это вердикт. Успех нечётен, неудача чётна. `STS$K_SUCCESS` равно 1, `STS$K_INFO` равно 3: оба означают «хорошо». `STS$K_WARN` равно 0, `STS$K_ERROR` равно 2, `STS$K_FATAL` равно 4: все означают «нехорошо». Всегда проверяйте через `1 & status`, а не через `== STS$K_SUCCESS`, потому что рутина может вернуть более конкретный успех — и потому что сравнение сломается именно в тот день, когда кто-нибудь его добавит. Тест библиотеки `test_status_convention` существует, чтобы держать каждый код по правильную сторону этой черты.

Условное значение — число, а его текст лежит в другом месте. Условное значение упаковывает средство, номер сообщения и серьёзность (глава 3). Слова принадлежат каталогу. Программа решает по числу, человек читает текст. Эти две вещи никогда не смешиваются.

Сообщения сигнализируются, а не печатаются. Рутина, нашедшая проблему, сообщает о ней через `$PUTMSG_FAO` и возвращает то же условное значение — одним выражением. Вызывающий видит диагностику в журнале и при этом держит статус в руке. Так делает код OpenVMS с `LIB$SIGNAL`, а выражение `return $PUTMSG_FAO(...)` — кратчайшая честная форма этого, какую позволяет Linux.

Вывод идёт через одну дверь. Библиотека пишет в единственный дескриптор, `g_logoutput` внутри модуля, задаваемый `__util$deflog` или `__util$setlogfd`. Нет ни перехватчика, ни обратного вызова, ни второго канала (когда-то он был, и его убрали). Программа, которой нужен чистый стандартный вывод, один раз вызывает `__util$setlogfd(STDERR_FILENO)`.

FAO, а не `printf`, в наших собственных исходных текстах. Форматы `printf` проверяются компилятором по типам, которые в этой библиотеке почти всегда одни и те же: 32-битные числа и строки. FAO добавляет то, что нужно системному программисту и чего нет у `printf`, и имеет единый смысл на любой платформе. Домашнее правило наших пакетов: сообщения и журнал идут только через `$LOG_FAO` и `$PUTMSG_FAO` (глава 13). Более старый `$LOG` с форматом `printf` сохранён для тех, кому он нужен, и описан здесь.

Счётные строки там, где важна длина. Тип `ASC` — это ASCIC из OpenVMS: один октет длины, затем до 255 октетов текста. Его дёшево копировать, безопасно сравнивать, и его можно построить на этапе компиляции макросом `$ASCDECL`. Командные таблицы главы 8 полны таких строк.

Статическая, а не разделяемая. Библиотека такого размера не заслуживает разделяемого объекта. Статический архив означает отсутствие сюрпризов с `LD_LIBRARY_PATH`, один файл для копирования и программу, которая работает на плате с голой корневой файловой системой. Пакет CMake устанавливает архив, заголовки и требование к использованию.

Движок не зависит от библиотеки. `starlet.c`, где живут движок FAO и рутины времени, не включает ни `<stdio.h>`, ни `<string.h>`, ни `<ctype.h>`; единственная нужная ему рутина из библиотеки Си — `time()`, и автономная (freestanding) сборка может заменить и её. Он сам преобразует цифры и сам считает календарь. Даже его диагностика по умолчанию выключена: его `$LOG` — пустой макрос, пока включающий не предоставит свой. Это позволяет перенести движок в загрузчик или в образ микропрограммы.

1.6 Кто его использует

Библиотека выросла из настоящих программ и до сих пор им служит. Каждая из них берёт у StarLet свой каталог сообщений, журнал или командную строку:

- **BATCH** — подсистема пакетных заданий: демон, очередь и клиент командной строки.
- **MSCP-Hub** — MSCP-сервер дисков для узлов OpenVMS; его диагностика идёт через `$PUTMSG_FAO`.
- **yaVTERM** — шифрованный терминал и передача файлов.
- **AlphaEmu** — эмулятор системы Alpha; ключевые слова его консоли и загрузки ищутся через `__util$lookup_key`.
- **SEARCH** — утилита поиска в манере `SEARCH` из OpenVMS; её таблица сообщений — разобранный пример каталога с вызовом `inimsg`.
- **yaTPU** — утилита обработки текста в манере TPU.
- **HELP** — читатель библиотек справки.

Когда это руководство нужно сверить с настоящей программой, открывайте именно эти. SEARCH, в частности, невелик и держит каталог в одной аккуратной таблице.

1.7 План обучения

Библиотеку задумано читать. Она мала, это простой Си, и каждая рутина в своей шапке говорит, что она делает. Главы следуют порядку, в котором идеи опираются друг на друга:

1. Глава 2: запустите первую программу и увидите сообщение в виде `%HELLO-I-GREET`.
2. Глава 3: добавьте своё сообщение; узнайте, что делает младший бит.
3. Глава 5: поиграйте с FAO, пока директивы не окажутся у вас в пальцах; всё остальное печатает через него.
4. Главы 4 и 6: напишите небольшую утилиту, которая читает файл, обрезает строки и пишет их в журнал.
5. Главы 7 и 8: дайте ей командную строку с глаголом и двумя квалификаторами.
6. Главы 9-11, когда понадобятся очередь, время или пакет.
7. Глава 13, а затем `utility_routines.c` с самого начала. Его история изменений — краткая история ремесла: каждая запись говорит, что было не так и почему это изменили.

А инженеру, знавшему OpenVMS в её времена: имена прежние, и, по большей части, прежние и идеи. Пожалуйста, прочтите также приложения C и D. Библиотеке, честной о своих углах и о том, что она изменила, можно доверять.

Глава 2. Начало работы

2.1 Что понадобится

Компилятор Си11 (GCC или Clang), CMake 3.10 или новее и среда сборки Linux, Windows (MinGW-w64) или Android. Библиотека написана на чистом Си; в её строке `project()` стоит `LANGUAGES C`, поэтому машина без компилятора C++ настраивается без жалоб.

2.2 Сборка

Из корня исходных текстов:

```
cmake -B build
cmake --build build
ctest --test-dir build
```

Первая строка выбирает каталог сборки. Вторая собирает статическую библиотеку, цель CMake **`starlet`**, и тестовые программы. Третья запускает тесты: двадцать пять прогонов, и все должны пройти.

Если тип сборки не указан, CMake собирает **Release**. Это сделано намеренно, и у этого есть следствие, которое стоит знать сразу: сборка Release определяет `NDEBUG`, а это вырезает собственный вывод `$TRACE` библиотеки и трассировку разборщика команд. Если трассировка нужна, попросите её:

```
cmake -B build-debug -DCMAKE_BUILD_TYPE=Debug
```

Сборка Debug определяет также `_DEBUG`, `__TRACE__` и `DEBUG`.

2.3 Установка в виде пакета

```
cmake --install build --prefix /usr/local
```

Это устанавливает архив `libstarlet.a`, четыре открытых заголовка (`utility_routines.h`, `starlet.h`, `cli_routines.h`, `avproto.h`) и пакет CMake в `lib/cmake/StarLet`. Другой проект затем пользуется им без `add_subdirectory()` и без `FetchContent`:

```
cmake_minimum_required(VERSION 3.10)
project(hello LANGUAGES C)

find_package(StarLet REQUIRED)

add_executable(hello main.c)
target_link_libraries(hello PRIVATE StarLet::starlet)
```

Если вы установили в личный префикс, скажите CMake, где искать: `-DCMAKE_PREFIX_PATH=$HOME/stuff`. Импортируемая цель `StarLet::starlet` приносит с собой каталог заголовков. В Windows она приносит также `ws2_32`.

Если вы предпочитаете держать исходные тексты внутри собственного дерева, `add_subdirectory(external/starlet)` даст ту же цель под тем же именем, `StarLet::starlet`. В этом случае правила пакета и CPack у StarLet пропускаются, так что `cpack` в вашем дереве соберёт *ваш* пакет.

Значение имеет и собственный тип сборки потребителя. `NDEBUG` у StarLet определяется при компиляции библиотеки, но макросы `$LOG`, `$LOG_FAO`, `$PUTMSG_FAO` и `$TRACE` раскрываются в *вашем* коде, так что именно *ваша* единица трансляции решает, понесёт ли запись модуль, рутину и строку.

2.4 Без CMake

Четыре исходных файла и компилятор:

```
cc -std=gnu11 -O2 -c utility_routines.c avproto.c cli_routines.c starlet.c
cc *.o -o your_app
```

Добавьте `-DNDEBUG` для тихой сборки. `sh tests/run.sh` собирает и запускает большую часть тестов тем же способом, уважая `CC` и `CFLAGS`.

2.5 Первая программа

Ниже — законченная программа. Она объявляет средство, два сообщения, регистрирует каталог и использует `$LOG_FAO` и `$PUTMSG_FAO`. Сохраните её как `hello.c`.

```
#define __MODULE__  "HELLO"
#define __FAC__     "HELLO"

#include    <stdlib.h>
#include    "utility_routines.h"
#include    "starlet.h"

#define HELLO$K_FACNO   0x0200

#define HELLO$_GREET    ($DEFFAC(HELLO$K_FACNO) | $DEFMSG(1) | $DEFSEV(STS$K_INFO))
#define HELLO$_NOFILE   ($DEFFAC(HELLO$K_FACNO) | $DEFMSG(2) | $DEFSEV(STS$K_ERROR))

#define $HELLOREC(a_sts, a_txt) \
    { .sts = (int) (a_sts), .textl = (unsigned char) (sizeof(a_txt) - 1), .text = a_txt,
 ↪  .mnemonic = #a_sts }

static  EMSG_RECORD s_msgtab [] = {
    $HELLOREC(HELLO$_GREET, "Hello, !AZ!! You are visitor number !UL"),
    $HELLOREC(HELLO$_NOFILE, "cannot open !AZ, errno=!SL")
    };

static  EMSG_RECORD_DESC    s_msgdsc = {
    .msgnr      = (unsigned) $ARRSZ(s_msgtab),
    .facno      = HELLO$K_FACNO,
    .msgrec     = s_msgtab,
    .facname    = __FAC__
    };

int main    (int argc, char *argv[])
{
int l_sts;

    __util$inimsg(&s_msgdsc);

    $LOG_FAO(STS$K_INFO, "started with !UL argument!%S", argc - 1);

    l_sts = $PUTMSG_FAO(HELLO$_GREET, "Vasya", 7);
    l_sts = $PUTMSG_FAO(HELLO$_NOFILE, "/no/such", 2);

    return  (1 & l_sts) ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Соберите и запустите её:

```
cc -std=gnu11 -I/usr/local/include hello.c /usr/local/lib/libstarlet.a -o hello
./hello a b
```

Она печатает три записи и завершается со статусом 1, потому что последнее сообщение было ошибкой:

```
31-10-2026 17:53:13.972 2702691 [HELLO\main:34] %HELLO-I:  started with 2 arguments
31-10-2026 17:53:13.972 2702691 [HELLO\main:36] %HELLO-I-GREET, Hello, Vasya! You are visitor number 7
31-10-2026 17:53:13.972 2702691 [HELLO\main:37] %HELLO-E-NOFILE, cannot open /no/such, errno=2
```

2.6 Что сделала программа

Два define. `__MODULE__` называет исходный модуль, он показывается в части записи `[module\routine:line]`. `__FAC__` — имя средства, которое идёт после `%`. Если не определить `__FAC__`, заголовок подставит `"UTILS"` — средство самой библиотеки, — и ваши записи будут выдавать себя за её записи. Определяйте оба до включения заголовка.

Порядок включения. Сначала `utility_routines.h`, затем `starlet.h`, как в примере. Начиная с 1.7.0 порядок этих двух не важен: `starlet.h` сам включает `utility_routines.h`, а старые запасные определения серьёзностей, ломавшие `enum`, когда заголовки шли в обратном порядке, убраны. `starlet.h` нужен для `$LOG_FAO` и `$PUTMSG_FAO`, которые приводят каждый параметр к его типу `fao_prm_t`. `cli_routines.h` и `avproto.h` по-прежнему берут свои типы из `utility_routines.h` и должны идти после него.

Макрос записи. `$HELLOREC` заполняет одну `EMSG_RECORD`: условное значение, длину текста (ASCIC, как в OpenVMS), сам текст и мнемонику, взятую из `#a_sts` — превращённого в строку имени `"HELLO$_GREET"`. `__util$inimsg` обрезает её до части после `$_`, так что запись печатается как `GREET`. Параметры макроса названы `a_sts` и `a_txt` намеренно: параметр с именем `sts` или `text` подменялся бы и внутри обозначенных инициализаторов `.sts = ...` и `.text = ...`.

Управляющая строка. `"Hello, !AZ!! You are visitor number !UL"` — это FAO. `!AZ` берёт строку с нулём в конце, `!UL` — беззнаковое длинное слово, а `!!` — буквальный восклицательный знак. Одинокий `!`, за которым идёт пробел, буквальным *не* является: это неизвестная директива, которую движок проглатывает (вместе с пробелом), устанавливая статус `FAO$K_BADDIR`. `!` в самом конце строки — та же ошибка, и сканер останавливается на завершающем нуле, а не читает дальше. Автор этого руководства выяснил это на собственной шкуре.

Параметры и возвращаемое значение. `$PUTMSG_FAO(HELLO$_GREET, "Vasya", 7)` принимает параметры как есть: макрос приводит каждый к `fao_prm_t`, 64-битному беззнаковому целому, и считает их. Он возвращает то условное значение, которое ему дали. У `HELLO$_NOFILE` серьёзность `STS$K_ERROR` (2), поэтому младший бит сброшен, и программа завершается неудачей. `$LOG_FAO` возвращает серьёзность, поэтому `return $LOG_FAO(STS$K_ERROR, ...)` сообщает и возвращает в одной строке.

Две формы записи. Без `NDEBUG`, как выше:

```
DD-MM-YYYY HH:MM:SS.fff <tid> [MODULE\routine:line] %FAC-S:  text
```

для `$LOG_FAO` и `... [MODULE\routine:line] %FAC-S-MNEMONIC, text` для `$PUTMSG_FAO`. С `-DNDEBUG` часть в скобках исчезает, и у записи `$LOG_FAO` после двоеточия один пробел, а не два:

```
31-10-2026 17:53:14.288 2702736 %HELLO-I: started with 0 arguments
31-10-2026 17:53:14.288 2702736 %HELLO-I-GREET, Hello, Vasya! You are visitor number 7
```

Глава 4 разбирает части записи одну за другой.

Глава 3. Условные значения и сообщения

3.1 Устройство условного значения

Условное значение — беззнаковое 32-битное целое из трёх полей, расположенных так же, как в OpenVMS:

```
 31                16 15                           3 2      0
+--------------------+-------------------------------+--------+
|  Facility           |  Message number              |Severity|
|  16 bits            |  13 bits                     | 3 bits |
+--------------------+-------------------------------+--------+
```

(В OpenVMS у средства 12 бит, а над ним четыре управляющих бита. StarLet отдаёт всю верхнюю половину номеру средства, потому что программе для Linux управляющие биты ни к чему.)

Макрос	Что делает
`$DEFFAC(n)`	Помещает номер средства в старшую половину: `n << 16`
`$DEFMSG(n)`	Помещает номер сообщения в биты 3-15, с маской до 13 бит: `(n << 3) & 0xFFF8`
`$DEFSEV(s)`	Берёт три младших бита серьёзности
`$FAC(c)`	Извлекает средство: `c >> 16`
`$MSG(c)`	Извлекает номер сообщения: `(c >> 3) & 0x1FFF`
`$SEV(c)`	Извлекает серьёзность: `c & 7`
`$STS_ERROR(c)`	Истинно для `STS$K_ERROR` и `STS$K_FATAL`

`$FAC`, `$MSG` и `$SEV` — точные обратные операции к макросам `$DEF...`. Так было не всегда: до версии 1.6.3 `$MSG` забывал отсекать средство. Теперь `test_status_convention` закрепляет раскладку поле за полем.

Номер сообщения изменяется от 1 до 8191. Ноль лучше не трогать.

3.2 Серьёзности и младший бит

Константа	Значение	Буква	Младший бит	Читается как
`STS$K_WARN`	0	W	0	не успех
`STS$K_SUCCESS`	1	S	1	успех
`STS$K_ERROR`	2	E	0	неудача
`STS$K_INFO`	3	I	1	успех
`STS$K_FATAL`	4	F	0	неудача
`STS$K_UNDEF`	5	?	1	(ограничитель для входа вне диапазона: печатается как `?`; рутины журнала возвращают для него `STS$K_ERROR`, а не 5)

Это назначение VMS, и причина у него хорошая. Серьёзность ноль, WARNING, *чётна*: предупреждение не является успехом, и программа, проверяющая `1 & status`, не примет его за успех. Она всё же может решить, что конкретное предупреждение безвредно, — но обязана сказать это явно. INFO нечётна: информационное сообщение не проблема.

```
status = __util$base64(src, srclen, dst, dstsz, &dstlen);

if ( !(1 & status) )
    return  $LOG_FAO(STS$K_ERROR, "base64 of !UL octets failed", srclen);
```

Никогда не пишите `if ( status == STS$K_SUCCESS )`. Рутина вправе вернуть другое нечётное значение как более конкретный успех, и в тот день ваше сравнение молча перестанет работать.

3.3 Коды, которые возвращает библиотека

В большинстве случаев рутины возвращают простые серьёзности. У некоторых есть собственные коды:

Константа	Значение	Где	Смысл	Младший бит
`FAO$K_NEGTIM`	0x08	время	Результат операции со временем был бы отрицательным	0
`FAO$K_BUFOVF`	0x0A	FAO, `__util$asctim`	Вывод усечён	0
`FAO$K_IVTIME`	0x0C	`__util$bintim`, время	Недопустимая строка времени или недопустимое сочетание	0
`FAO$K_BADDIR`	0x12	FAO	Неизвестная, неверно сформированная или оборванная директива	0
`FAO$K_NOPRM`	0x1A	FAO	Управляющей строке нужно больше параметров, чем дано	0
`FAO$K_FLDOVF`	0x22	FAO	Текст поля `!n<...!>` был длиннее 512 октетов и обрезан	0
`UTIL$S_INVARG`	0x102	очереди, `__util$inimsg`	Аргумент NULL или непригодный	0
`UTIL$S_INQUE`	0x202	`$INSQTAIL`, `$INSQHEAD`	Элемент уже находится в другой очереди	0
`UTIL$S_NOLOCK`	0x302	`$INSQTAIL`, `$INSQHEAD`	Не удалось взять блокировку очереди	0
`UTIL$S_DUPFAC`	0x400	`__util$inimsg`	Каталог этого средства уже зарегистрирован	0

Все они чётны. Коды FAO и времени следуют раскладке OpenVMS (серьёзность в битах 0-2, номер сообщения выше, без средства), и начиная с 1.7.0 их номера стабильны. `FAO$K_BADDIR` раньше был равен 6; программу, которая сравнивала с числом, а не с именем, необходимо пересобрать. `UTIL$S_NOLOCK` раньше делил значение с `UTIL$S_INQUE`; теперь у каждого отказа свой код. Два кода FAO когда-то были нечётными, то есть читались как успех; в X.01-10 их перенумеровали, оставив пояснение в комментарии. Когда вызов FAO находит несколько ошибок, статусом становится **первая**.

3.4 Определение средства и его сообщений

Средство (facility) — это программа или библиотека с номером. Выберите такой, которого не используют другие средства в том же процессе. Сама библиотека ничего не регистрирует, но имя её средства по умолчанию — `"UTILS"`.

Удачное соглашение, взятое из SEARCH:

```
#define MYFAC$K_FACNO   0x0301

#define $MYSTS(msg, sev)    ( $DEFFAC(MYFAC$K_FACNO) | $DEFMSG(msg) | $DEFSEV(sev) )

#define MYFAC$_OPENIN   $MYSTS(3, STS$K_ERROR)
#define MYFAC$_DONE $MYSTS(4, STS$K_INFO)
```

Имя условия — `FAC$_IDENT`: средство, знак доллара, подчёркивание, опознаватель сообщения. Эта форма важна, потому что `__util$inimsg` находит мнемонику после `$_`.

Номер сообщения следует использовать *один раз на средство*. Каталог ищет сообщения по средству и номеру и игнорирует серьёзность (см. раздел 3.6).

3.5 Каталог

Каталог — это массив `EMSG_RECORD` и дескриптор `EMSG_RECORD_DESC`.

```
typedef struct __emsg_record__ {
    int     sts;        /* the condition value */
    struct { unsigned char textl; char text[254]; };   /* the FAO text, an ASCIC */
    const char  *facname;   /* stamped by __util$inimsg() */
    const char  *mnemonic;  /* optional: "FAC$_NAME", trimmed by __util$inimsg() */
} EMSG_RECORD;

typedef struct __emsg_record_desc__ {
    struct __emsg_record_desc__ *link;  /* used by the library */
    unsigned    msgnr;      /* the number of records */
    unsigned    facno;      /* the facility number */
    struct __emsg_record__ *msgrec; /* the array */
    const char  *facname;   /* the facility name */
} EMSG_RECORD_DESC;
```

Текст записи ограничен 254 октетами и является управляющей строкой FAO — не форматом `printf`, — поэтому в нём используются `!AZ`, `!UL` и прочие директивы. Все они приведены в главе 5.

Массив заполняется обозначенными инициализаторами, как в первой программе. Заголовок сообщения по обычаю VMS — его опознаватель: `"NOFILE, no file matches !AZ"`. Если запись несёт мнемонику (как её сделал `$HELLOREC`), опознаватель добавляет сигнализирующая рутина, и текст *не* должен его повторять. Если у записи нет мнемоники (SEARCH держит опознаватель в тексте), опознаватель — часть текста. Допустимы оба способа. В одном каталоге используйте один.

Зарегистрируйте каталог один раз, при запуске:

```
unsigned    __util$inimsg   (EMSG_RECORD_DESC *msgdsc);
```

`__util$inimsg` делает три вещи. Она **сортирует массив на месте** через `qsort`, чтобы поиск мог быть двоичным: поэтому массив **не должен быть `const`**. Она проставляет имя средства в каждую запись и обрезает мнемоники. И она связывает дескриптор в список.

Она возвращает `STS$K_SUCCESS`, если всё прошло хорошо, `UTIL$S_INVARG`, если дескриптор равен NULL, и `UTIL$S_DUPFAC`, если каталог с тем же номером средства уже зарегистрирован (в этом случае ничего не меняется). Оба отказа имеют сброшенный младший бит. Начиная с 1.7.0 вызов потокобезопасен: регистрации сериализуются, а список публикуется лишь тогда, когда дескриптор готов целиком, так что поток, ищущий в это время сообщение, видит либо старый список, либо новый, но не половину. Всё же аккуратнее регистрировать до запуска потоков.

Чтобы найти запись, не печатая её:

```
unsigned    __util$getmsg   (unsigned sts, EMSG_RECORD **outmsg);
```

Возвращается `STS$K_SUCCESS` и адрес записи либо `STS$K_ERROR`, когда такого средства или сообщения нет.

3.6 Сигнализация сообщения

```
unsigned    __util$putmsg_fao   (unsigned sts, int prmcnt, ...);
unsigned    __util$putmsgd_fao  (unsigned sts, const char *mod, const char *func,
 ↪  unsigned line,
                int prmcnt, ...);
```

Напрямую их не вызывают. Пользуются макросом:

```
$PUTMSG_FAO(sts [, parameter ...])
```

Макрос раскрывается в `__util$putmsgd_fao` без `NDEBUG` и в `__util$putmsg_fao` с ним. Он приводит каждый параметр к `fao_prm_t` и вычисляет число по списку (от нуля до шестнадцати параметров). Рутина:

1. находит запись по средству и номеру сообщения — *игнорируя биты серьёзности*, так что одну и ту же запись можно сигнализировать как INFO в одном месте и как WARNING в другом;
2. пишет обёртку `%FACILITY-S-MNEMONIC,`, беря букву серьёзности из условного значения **вызывающего** (или `%FACILITY-S-`, если у записи нет мнемоники);
3. форматирует текст через `__util$faol`;
4. добавляет перевод строки и пишет запись с обычным префиксом одним `write()`;
5. возвращает то условное значение, которое ей дали.

Итак, сообщение серьёзности ERROR печатается как `-E-` и возвращается чётным числом. Вызывающий может написать

```
if ( 0 > (l_rc = mkdir(l_path, 0755)) && (errno != EEXIST) )
    return  $PUTMSG_FAO(MYFAC$_MKDIRERR, l_path, errno);
```

и получить отчёт и статус в одном выражении. Ветеран увидит в этом облик `LIB$SIGNAL`, с той разницей, что ничего не раскручивается: рутина только сообщает и возвращает.

Неизвестное условие для печатающей рутины не ошибка. Если запись не найдена, она печатает строку по умолчанию с расшифрованными числами и всё равно возвращает условие:

```
%NONAME-E-NOMSG, Message number: 03010012 [fac: 0x301/769, sev: 0x2/2, msgno: 0x2/2]
```

(Как и любое другое сообщение библиотеки, строка по умолчанию начинается с `%`; до 1.7.0 это было не так.) Увидев `NOMSG` в журнале, знайте: сигнализировано сообщение, каталог которого так и не зарегистрирован, — или номер которого неверен.

Старая пара, `$PUTMSG` и `__util$putmsg`, находит запись тем же способом, но считает её текст форматом `printf` и не пишет обёртки. Они сохранены для старых вызывающих. Новый код использует `$PUTMSG_FAO`.

Замечание для ветерана. Настоящий `SYS$PUTMSG` принимает *вектор сообщений*, может сцеплять несколько сообщений и вызывает рутину действия. Эта принимает условное значение и его параметры, и это всё, что она делает. Если вы скучаете по вектору сообщений, вы в хорошей компании.

3.7 Получение текста без печати

Вызывающий, которому нужен отформатированный текст в буфере, берёт запись через `__util$getmsg` и форматирует через `__util$faol`, передав `l_rec->text` в качестве управляющей строки. Результат — счётная последовательность октетов **без завершающего нуля** (глава 5).

Глава 4. Журнал

4.1 Рутины с первого взгляда

Макрос	Рутина	Формат	Назначение
`$LOG_FAO(sev, ctl, ...)`	`__util$log_fao`, `__util$logd_fao`	FAO	**Рекомендуется.** Одна запись в журнал
`$LOG(sev, fmt, ...)`	`__util$log`, `__util$logd`	`printf`	Более старая форма
`$PUTMSG_FAO(sts, ...)`	`__util$putmsg_fao`, `__util$putmsgd_fao`	FAO, из каталога	Сообщение по условному значению (глава 3)
`$TRACE(fmt, ...)`	`__util$trace`	`printf`	Отладочная трассировка; при `NDEBUG` вырезается
`$IFTRACE(cond, fmt, ...)`	`__util$trace`	`printf`	Трассировка, охраняемая условием
`$DUMPHEX(ptr, len)`	`__util$dumphex`	-	Дамп памяти в hex и ASCII
-	`__util$log2buf`	`printf`	Та же запись, что у `$LOG`, но в буфер вызывающего
-	`__util$out`	`printf`	Сырая строка без префикса

Макросы, у которых есть отладочный вариант (`__util$logd`, `__util$putmsgd_fao`), — те, что добавляют модуль, рутину и строку. Какой вариант выбран, решает `NDEBUG` **в единице трансляции, раскрывающей макрос**.

4.2 Запись

Префикс везде одинаков:

```
31-10-2026 16:22:45.701 1951513 [HELLO\main:34] %HELLO-I:  text
|          |            |       |                |
|          |            |       |                +-- the envelope: %facility-severity
|          |            |       +-- module\routine:line   (without NDEBUG only)
|          |            +-- thread number, six columns
|          +-- time of day, with a fraction of the second
+-- date, DD-MM-YYYY, in the local time zone
```

(Слева направо: дата в виде ДД-ММ-ГГГГ по местному часовому поясу; время суток с долей секунды; номер потока, шесть колонок; модуль\рутина:строка — только без NDEBUG; обёртка %средство-серьёзность.)

Источник	После префикса	Примечания
`$LOG_FAO`, `$LOG` (отладка)	`[MOD\func:line] %FAC-S: text`	Два пробела после двоеточия
`$LOG_FAO`, `$LOG` (NDEBUG)	`%FAC-S: text`	Один пробел
`$PUTMSG_FAO`	`[MOD\func:line] %FAC-S-IDENT, text`	Без NDEBUG; с ним скобок нет
`__util$log2buf`	`%FAC-S:text`	Без пробела, без перевода строки; в ваш буфер
`$TRACE`	`[MOD\func:line]`, пробелы до колонки 72, текст	С NDEBUG — ничего
`$DUMPHEX`	`[func:line] Dump of N octets follows:`	Затем строки `\t+0000:  48 65 ...`

Что стоит знать:

- **Доля секунды — миллисекунды**, от 000 до 999, на любой платформе. (До 1.7.0 это были наносекунды, сдвинутые вправо на 20 бит, что давало значения от 000 примерно до 953, а в Windows печатало шесть цифр.) Часы — грубые, `CLOCK_REALTIME_COARSE`: дешёвые и точные до нескольких миллисекунд, так что последняя цифра измерением не является.
- **Дата и время — местные**, из `localtime_r`. (Директивы FAO `!%D` и `!%T` печатают UTC. Объяснение в главах 5 и 10.)
- **Номер потока** — номер ядра, получаемый один раз на поток. После `fork()` потомок наследует кэшированный номер; если это важно потомку, ему следует выполнить re-exec или сбросить номер.
- **У записи есть верхний предел в 2047 октетов** (и перевод строки). Более длинные обрезаются внутри буфера; `test_log_overflow` проверяет, что завершающий перевод строки всегда попадает внутрь.
- **Один `write()` на запись.** Буфер строится, затем пишется целиком. Два потока не перемешиваются внутри строки. (`$DUMPHEX` пишет по строке за раз.)

4.3 $LOG_FAO

```
unsigned __util$log_fao (const char *fac, unsigned sev, const char *ctl, int prmcnt, ...);
```

Вы пишете:

```
$LOG_FAO(STS$K_WARN, "queue !AZ holds !UL entr!%S", l_name, l_count);
```

а макрос подставляет средство (`__FAC__`), модуль, рутину и строку (без `NDEBUG`), приведения типов и счёт. До шестнадцати параметров. Серьёзность — одна из `STS$K_*`; печатается буква `W`, `S`, `E`, `I` или `F`. Рутина возвращает серьёзность, поэтому

```
return  $LOG_FAO(STS$K_ERROR, "no room for !UL octets", l_size);
```

сообщает и возвращает чётное число в одной строке. А поскольку серьёзность INFO равна 3,

```
return  $LOG_FAO(STS$K_INFO, "all done");
```

возвращает нечётное: успех. Это изящная идиома, и сама библиотека ею пользуется.

Рутина возвращает записанную в журнал серьёзность, так что значение всегда годится для возврата.

Если форматировщик FAO нашёл в управляющей строке ошибку (неверная директива, мало параметров, поле или вывод обрезаны), запись всё равно пишется, а в её конце стоит короткая метка **`[FAO BADDIR]`**, `[FAO NOPRM]`, `[FAO FLDOVF]` или `[FAO BUFOVF]` (другой статус виден как `[FAO 0x...]`), так что опечатка заметна сразу. Возвращаемое значение не меняется. `$PUTMSG_FAO` делает то же. (До 1.7.1 статус отбрасывался.) Серьёзность вне диапазона 0..4 печатается как `?`, а рутина возвращает **`STS$K_ERROR`** (младший бит сброшен): неопределённая серьёзность — ошибка вызывающего и не должна читаться как успех (до 1.7.1 возвращалось 5, у которого младший бит установлен). Бит syslog (раздел 4.6) снимается *до* этой проверки, поэтому вызов, помеченный для syslog, тоже возвращает обычную серьёзность; до 1.7.0 любой помеченный вызов возвращал 5, что читается как успех. То же верно для `$LOG` — и в отладочной единице, и в единице с `NDEBUG`.

`$LOG` — то же самое со строкой формата `printf` и аргументами `printf`. Пользуйтесь им лишь там, где параметры FAO не помогли бы.

4.4 Куда идёт журнал

По умолчанию — в стандартный вывод. Изменить это можно двумя рутинами.

```
int __util$deflog    (const char *logfile, const char *sloghost);
int __util$setlogfd  (int fd);
```

__util$deflog открывает `logfile` для добавления (создавая его с режимом `rw-r--r--`, с close-on-exec), делает его журналом и *вдобавок* через `dup2` накрывает им стандартный вывод и стандартную ошибку, так что случайный `printf` попадёт в тот же файл. Затем она записывает `Log file '...' has been opened`. NULL или пустое имя оставляет всё как есть и завершается успешно. При повторном вызове она закрывает файл, открытый прежде. Для демона это правильный вызов.

Второй аргумент, `sloghost`, — текст адреса IPv4 для сборщика syslog по UDP, порт 514. Текст, не являющийся адресом IPv4, регистрируется в журнале и игнорируется. **__util$deflog лишь запоминает этот адрес.** Ни одна рутина журнала ничего ему не отправляет, какова бы ни была серьёзность и бит syslog: `STS$M_SYSLOG` идёт в локальный `syslog(3)` (раздел 4.6). Сборщик — адресат `__util$syslog`, которую вы вызываете сами. Передайте NULL, если сборщика нет.

__util$setlogfd направляет журнал в уже открытый дескриптор и больше ничего не трогает: файл не открывается, `dup2` нет, ничего не захватывается. Классическое применение — фильтр, чей стандартный вывод несёт данные:

```
#include <unistd.h>

__util$setlogfd(STDERR_FILENO);
```

С этого момента `$LOG`, `$PUTMSG_FAO` и `$TRACE` идут в стандартную ошибку, а данные остаются чистыми. Для отрицательного дескриптора возвращается `STS$K_ERROR`. Дескриптор — ваш; библиотека никогда не закрывает его и никогда не ротирует.

4.5 Ротация

Демон, который пишет журнал месяцами, нуждается в том, чтобы журнал держали в рамках. Это делают две рутины. Вызывайте любую из них время от времени — раз в несколько секунд вполне достаточно.

```
int __util$rotatelogfile (size_t limit, unsigned keep);
int __util$rewindlogfile (size_t limit);
```

__util$rotatelogfile(limit, keep) — хорошая. Когда файл, открытый `__util$deflog`, вырос до `limit` октетов, она переименовывает `file.<keep-1>` в `file.<keep>` и так далее вниз до `file` в `file.1` (самый старый отбрасывается), затем открывает свежий `file` на том же самом дескрипторе, а с ним — и на стандартном выводе и ошибке. Ничего не копируется: переименование — изменение метаданных, поэтому медленный диск не задерживает программу, и ничего не теряется. Запись, которую другой поток пишет в это время, попадает в конец старого файла или в начало нового, но никогда не рвётся. `keep` изменяется от 0 до 99; 0 усекает на месте. Предел, равный нулю, журнал в стандартный вывод или дескриптор, заданный через `__util$setlogfd`, означают «делать нечего». Ротирует только один поток за раз; остальные видят, что она идёт, и продолжают работу.

__util$rewindlogfile(limit) — грубая: за пределом она перематывает и усекает файл. Старые записи теряются. Размер проверяется один раз, в позиции дескриптора. Оставьте её для утилит, где журнал — черновик.

```
__util$deflog("/var/log/myd.log", NULL);

for ( ;; )
    {
    /* ... the work of the daemon ... */
    __util$rotatelogfile(10 * 1024 * 1024, 5);  /* 10 MB, keep 5 */
    }
```

4.6 Syslog и Logcat

Добавьте по OR `STS$M_SYSLOG` (16) к серьёзности, и запись *вдобавок* отправится в системный `syslog(3)`. Приоритет — `LOG_INFO` для серьёзности с установленным младшим битом (SUCCESS, INFO) и `LOG_ERR` для остальных. Это работает в системах, кроме Windows, для `$LOG` и `$LOG_FAO` — и в отладочной единице, и в единице с `NDEBUG`. (До 1.7.0 это делала только форма `NDEBUG`, а приоритет всегда выходил `LOG_INFO`, причём по ошибке подмешивался `LOG_PID`.) `__util$log2buf` пишет в буфер и никогда — в syslog.

`STS$M_SYSLOG` не питает сборщик UDP, названный в `__util$deflog`; чтобы отправить ему датаграмму, вызовите рутину сами:

```
unsigned __util$syslog (int fac, int sev, const char *tag, const char *msg, int msglen);
```

Она отправляет `<fac+sev> tag: msg` на порт 514. Если сборщик не задан, она ничего не делает и возвращает `STS$K_SUCCESS`; собственной диагностики она не пишет. Она возвращает `STS$K_ERROR`, если не удалось создать сокет или не удалась отправка. Сокет создаётся один раз, при первом использовании, под охраной, через которую потоки могут состязаться.

В Android определите `ANDROID_LOGCAT` при компиляции библиотеки, чтобы каждая запись писалась ещё и в `logcat` (глава 12).

4.7 Трассировка и дамп

```
$TRACE("opening %s, flags=%#x", "/etc/hosts", 2);
```

`$TRACE` принимает формат `printf`. `$TRACE` и `$IFTRACE` с `NDEBUG` превращаются в ничто, так что трассировка ничего не стоит в готовой сборке. С `NDEBUG` аргументы даже не вычисляются; `$IFTRACE` держит своё условие внутри `sizeof`, так что переменная, используемая только в трассировке, не вызывает предупреждения «не используется».

`$DUMPHEX(ptr, len)` пишет дамп до 65535 октетов, по шестнадцать в строке:

```
31-10-2026 16:24:27.227 1964971 [main:8] Dump of 18 octets follows:
    +0000:  48 65 6c 6c 6f 2c 20 53 74 61 72 4c 65 74 21 0a   Hello, StarLet!.
    +0010:  01 02                                             ..
```

`__util$faohex(src, srclen, out, outsz, &outlen)` форматирует одну такую строку (не более 16 октетов источника; hex и текст, нужно 67 октетов места) в ваш собственный буфер, если вы хотите писать дамп не в журнал. Она возвращает записанную длину либо 0, если буфер слишком мал или аргумент равен NULL; начиная с 1.7.0 `outlen` может быть NULL.

`$TRACE` — инструмент отладки. Для того, о чём нужно сообщать всегда, используйте `$LOG_FAO`.

4.8 Записи в буфер

```
unsigned __util$log2buf (void *out, int outsz, int *outlen, const char *fac, unsigned sev,
            const char *fmt, ...);
```

Форматирует запись в точности как `$LOG`, но в `out` — без перевода строки, с завершающим нулём, всегда внутри `outsz`, с длиной в `*outlen`. Она нужна, когда текст записи требуется как строка. Строка имеет вид `date time tid %FAC-S:text`. Как и `$LOG`, она возвращает записанную в журнал серьёзность.

`__util$out(fmt, ...)` пишет сырую строку в дескриптор журнала: без префикса, с добавленным переводом строки. Ею пользуется генератор справки разборщика команд, поэтому справка идёт туда же, куда и журнал.

Глава 5. FAO — форматировщик

5.1 Зачем FAO

SYS$FAO — одна из лучших небольших идей OpenVMS. *Управляющая строка* описывает вывод; *параметры* — это данные; ничто не зависит от того, проверит ли компилятор формат на соответствие типам, потому что директива в точности говорит, какого размера каждый параметр. Одни и те же директивы работают в записи журнала, в сообщении, на экране и в поле протокола. Движок StarLet, __util$fao и __util$faol, — его переносимая переработка на простом Си. Он не использует ни <stdio.h>, ни <string.h>, ни <ctype.h>.

```
int __util$fao  (const char *ctlstr, unsigned short *outlen, fao_desc_t *outbuf, ...);
int __util$faol (const char *ctlstr, unsigned short *outlen, fao_desc_t *outbuf,
         const fao_prm_t *prmlst, int prmcnt);
```

outbuf — дескриптор над вашим буфером, небольшая структура fao_desc_t из семейства dsc$descriptor_s OpenVMS (слово длины, два неиспользуемых байта типа и класса, адрес). Инициализатор для массива char — $FAO_DESC(buf). В *outlen помещается длина результата. **Результат не завершается нулём**, как и в OpenVMS; добавьте его сами, если он нужен.

Возвращаемое значение — условное значение: STS$K_SUCCESS либо **первая** найденная ошибка; текст в любом случае создаётся. FAO$K_BUFOVF означает, что вывод усечён до размера буфера; FAO$K_BADDIR — что директива неизвестна, неверно сформирована или оборвана концом строки; FAO$K_NOPRM — что управляющей строке требовалось больше параметров, чем ей дали; FAO$K_FLDOVF — что текст поля !n<...!> был длиннее 512 октетов и обрезан. STS$K_ERROR означает аргумент NULL. Движок никогда не пишет за конец буфера и никогда не читает дальше завершающего нуля управляющей строки. outlen может быть NULL.

__util$faol принимает параметры в виде массива и числа. Директива, требующая больше параметров, чем есть в массиве, получает ноль, а статус — FAO$K_NOPRM. __util$fao принимает их списком переменной длины и не может знать, сколько вы передали: если управляющая строка просит больше, чем вы дали, поведение не определено, а FAO$K_NOPRM возникает лишь тогда, когда требуется более 64 параметров. Вот почему существуют макросы $LOG_FAO и $PUTMSG_FAO и вот почему они используют faol.

5.2 Параметры

Каждый параметр — `fao_prm_t`, беззнаковое 64-битное целое, **передаваемое по значению**. В нём передаётся что угодно: указатель, адрес строки, время, 64-битное число. С `__util$fao` каждый вы приводите сами:

```
char        l_buf [128];
fao_desc_t  l_dsc = $FAO_DESC(l_buf);
unsigned short  l_len;

__util$fao("!AZ has !UL file!%S", &l_len, &l_dsc, (fao_prm_t) "BACKUP", (fao_prm_t) 3);
```

С макросами вы пишете их как есть. Квадрослово передаётся по значению — и так уже с X.01-09.

Особенность для ветерана: в OpenVMS каждый параметр — это длинное слово и адрес. Здесь директива квадрослова читает все 64 бита параметра. Знаковое значение сначала приведите через его собственную ширину: `(fao_prm_t)(int32_t) -42`.

5.3 Директивы

Директива начинается с `!`. Между `!` и буквами можно поставить число — *счётчик повторений* или *ширину поля*, — либо `#`, который берёт число из списка параметров.

Строки

Директива	Параметр	Вывод
`!AZ`	Адрес строки с нулём в конце	Строка. Адрес NULL не печатает ничего
`!AS`	Адрес `fao_desc_t`	Текст, который описывает дескриптор
`!AC`	Адрес счётной строки (`ASC`)	Текст после октета длины
`!AD`	Два: длина, затем адрес	Столько октетов
`!AF`	Два: длина, затем адрес	Как `!AD`, но непечатаемый октет показывается как `.`

Ширина поля, как в `!12AZ`, дополняет пробелами справа или обрезает справа: `!12AZ` от `zero-term` даёт `zero-term`, а `!4AZ` даёт `zero`.

Числа. Вторая буква — размер: **B** байт, **W** слово, **L** длинное слово, **Q** квадрослово.

Директива	Вывод
`!UB !UW !UL !UQ`	Беззнаковое десятичное
`!SB !SW !SL !SQ`	Знаковое десятичное (значение расширяется знаком от своего размера)
`!XB !XW !XL !XQ`	Шестнадцатеричное, заглавными, с нулями до 2, 4, 8, 16 цифр
`!OB !OW !OL !OQ`	Восьмеричное, с нулями до 3, 6, 11, 22 цифр
`!ZB !ZW !ZL !ZQ`	Беззнаковое десятичное, с нулями до ширины поля (естественная ширина, если её нет)

С шириной `n` десятичное число выравнивается вправо в поле и дополняется пробелами; `!6UL` от 42 даёт `42`. Если число не помещается, поле заполняется звёздочками: `!2UL` от 12345 даёт `**`. Шестнадцатеричное или восьмеричное число, которое не помещается, обрезается *слева*: `!4XL` от `0x12345678` даёт `5678`. Таково правило OpenVMS, и оно легко удивляет.

Управление

Директива	Вывод
`!/`	Возврат каретки и перевод строки (CR LF, как в OpenVMS)
`!_`	Горизонтальная табуляция
`!^`	Перевод формата
`!!`	Буквальный `!`
`!%S`	Буква `s`, если последнее преобразованное число не равно 1
`!n*c`	Символ `c`, повторённый `n` раз (один раз, если `n` нет)
`!n(...!)`	Заключённые директивы, повторённые `n` раз

Директива	Вывод
`!n<...!>`	Заключённые директивы, отформатированные в поле из `n` октетов, дополненное пробелами или обрезанное справа. Текст внутри может быть до 512 октетов; больше — обрезается и устанавливается `FAO$K_FLDOVF`
`!n-`	Шаг назад на `n` параметров (на один, если `n` нет): следующая директива читает те же параметры снова
`!n+`	Пропустить `n` параметров

`!/` — это CR LF. Поток Unix ожидает просто LF, поэтому программы под Linux, печатающие строки, пишут `\n` сами. Рутины журнала добавляют `\n` за вас.

Группы вкладываются на глубину до четырёх. Незавершённая или слишком глубокая группа устанавливает `FAO$K_BADDIR`, как и директива, посередине которой кончается строка (`!`, `!A`, `!%`, `!3*`).

Время и дата. Параметр — время по значению: секунды эпохи UNIX. Решает знак:

Параметр	`!%D`	`!%T`
Положительный	`dd-MMM-yyyy hh:mm:ss.cc`, день с пробелом до двух знаков, месяц заглавными	`hh:mm:ss.cc`
Нуль	Текущее время. Саму эпоху показать нельзя; на секунду позже — можно	Текущее время
Отрицательный	Дельта в столько секунд: `dddd hh:mm:ss.cc`	`hh:mm:ss.cc`

Итак, время до 1970 года нельзя записать как абсолютное: отрицательное число — это всегда дельта. Год выше 9999 печатается полностью (`1-JAN-10000 00:00:00.00`); до 1.7.0 он читал за пределами своей таблицы цифр. Сотые доли всегда `.00`. Время — это UTC плюс константа `FAO$K_TZOFFSET`, фиксируемая при компиляции `starlet.c` (по умолчанию нуль). Это не местное время, чем оно и отличается от префикса записи журнала.

Адреса. Параметр — адрес сырых октетов. Адрес NULL печатает `(null)`.

Директива	Читает	Вывод
`!%M`	6 октетов	Адрес MAC, `00:1b:44:11:3a:b7`
`!%I`	4 октета	Адрес IPv4, `192.168.42.2`
`!%6`	16 октетов	Адрес IPv6 в каноническом виде RFC 5952, `2001:db8::1`; отображённый из IPv4 — как `::ffff:192.168.1.10`
`!%U`	16 октетов	UUID, `12345678-9abc-def0-1122-334455667788`

Текст — строчными буквами. Форма `!%6` побайтно совпадает с `inet_ntop()`, и `test_fao_addr` держит её в этом: самая длинная серия из двух и более нулевых групп превращается в `::`, при равенстве — самая левая, а одиночная нулевая группа никогда не сжимается.

5.4 Обзор настоящего вывода

Каждая строка ниже получена кодом (помощник `T(ctl, ...)`, вызывающий `__util$fao`):

```
!UL          42       -> [42]
!XL          255      -> [000000FF]
!5ZL         7        -> [00007]
!6UL|        42       -> [    42|]
!2UL         12345    -> [**]
!4XL         0x12345678 -> [5678]
!AC          "VMS-8"  -> [VMS-8]
!AD          2, "zero-term" -> [ze]
!12AZ|                -> [zero-term   |]
!UL file!%S  1        -> [1 file]
!UL file!%S  3        -> [3 files]
!8<abc!>|             -> [abc     |]
!UL !-!XL    255      -> [255 000000FF]
!#UL|        6, 9     -> [     9|]
!%M                   -> [00:1b:44:11:3a:b7]
!%D          1793462400 -> [31-OCT-2026 16:00:00.00]
!%D          -93784     -> [1 02:03:04.00]       a delta: one day, 2 h 3 min 4 s
!%D          253402300800 -> [ 1-JAN-10000 00:00:00.00]
!UL !UL      1 (one parameter given) -> [1 0]    status FAO$K_NOPRM (0x1A)
!QL                   -> [L]       status FAO$K_BADDIR (0x12)
Ab!A             -> [Ab]    status FAO$K_BADDIR (0x12): the string ends inside the directive
(a 16-octet text into an 8-octet buffer) -> [01234567]  status FAO$K_BUFOVF (0x0A)
```

Посмотрите на последние четыре. Неверная директива не останавливает движок: он устанавливает статус и идёт дальше. Параметр, которого не дали, читается как нуль. Слишком маленький буфер заполняется до краёв, и статус об этом говорит. Сообщается только первая ошибка; последующие не теряются, а просто не учитываются.

5.5 FAO в сообщениях и журналах

Всё в главах 3 и 4 использует этот движок. Его параметры передаёт макрос. Например

```
$LOG_FAO(STS$K_INFO, "client !%I:!UL connected from !%M", l_ip4, (unsigned) l_port,
↪  l_mac);
```

где `l_ip4` — массив `unsigned char[4]`, а `l_mac` — массив `unsigned char[6]`. Макрос приводит массивы к `fao_prm_t`, а для массива это адрес его первого октета.

Два предела макросной формы: не более шестнадцати параметров и отформатированное тело не более 2047 октетов.

5.6 Подводные камни

- **Одинокий `!` — это директива.** Чтобы напечатать восклицательный знак, пишите `!!`. `!`, которым кончается строка, — это `FAO$K_BADDIR`, и ничего за завершающим нулём не читается.
- **Квадрослова по значению.** `!UQ`, `!XQ` и `!SQ` читают параметр как есть.
- **Счёт ваш при `__util$fao`.** Макросы считают за вас; рутина с переменным числом аргументов — не может.
- **`!AZ` от указателя NULL ничего не печатает**, и `!AS`/`!AC` от NULL тоже; директивы адресов печатают `(null)`.
- **Нет хвостового нуля.** Вывод — счётная последовательность октетов.
- **Движок сообщает о своих ошибках молча.** Внутри `starlet.c` макрос `$LOG` пуст, так что неверная директива сообщается только через статус (`FAO$K_BADDIR`, `FAO$K_NOPRM`, …). `$PUTMSG_FAO` и `$LOG_FAO` этот статус не проверяют, поэтому опечатка в управляющей строке даёт испорченный вывод и ни одной жалобы. Проверьте свои сообщения один раз, просто посмотрев на них.
- **Время нуль — это «сейчас».** Отчёт о событии, случившемся в момент эпохи, покажет вместо него часы текущего момента. Отрицательное время — это дельта, а никогда не дата до 1970 года.

Глава 6. Строки и данные

6.1 Счётная строка

В OpenVMS строку часто носят как ASCIC: один октет длины, затем текст. Она дёшева, для поиска её конца не нужен просмотр, и она может содержать нулевой октет. `ASC` в StarLet — именно такая строка:

```
typedef struct _asc {
    unsigned char   len;        /* 0..255 */
    char        sts [255];  /* the text */
} ASC;
```

Макрос	Что делает
`$ASCDECL(a, "text")`	Объявляет и инициализирует `ASC` по имени `a` из литерала
`$ASCINI("text")`	Список инициализатора `len, "text"`; используйте внутри скобок в таблице
`$ASCLEN(p)` / `$ASCPTR(p)`	Длина и адрес текста у `ASC *`
`$ASC(p)`	Раскрывается в `len, text`: два аргумента, которых требует `"%.*s"`
`$ASCNIL`	`0, 0` — пустая запись

```
$ASCDECL(l_name, "counted");
char    l_z [64];

__util$asc2str(&l_name, l_z);
$LOG_FAO(STS$K_INFO, "name is !AC, length !UL", &l_name, (unsigned) $ASCLEN(&l_name));
```

Директива FAO `!AC` принимает адрес `ASC` напрямую.

Рутины:

Рутина	Что делает
`__util$str2asc(const char *src, void *dst)`	Копирует строку Си в `ASC` и завершает её; текст длиннее 254 октетов обрезается там, чтобы нуль остался на своём месте внутри `sts[]`; возвращает сохранённую длину
`__util$asc2str(ASC *src, char *dst)`	Копирует `ASC` в строку Си и завершает её; возвращает длину
`__util$cmpasc(a, b)`	Сравнивает две `ASC`: сначала разность длин, затем октеты
`__util$cmpasc_blind(a, b)`	То же без учёта регистра
`__util$cpyasc(src, dst)`	Копирует одну `ASC` в другую

Рутина	Что делает
`__util$strelem(src, sep, idx, dst)`	`idx`-й кусок (от 0) строки `src`, разбитой по `sep`, в `ASC`; возвращает его длину. Для отрицательного индекса или строки с меньшим числом кусков `dst` остаётся пустым, а результат равен 0

Помните, что здесь значит «сравнить»: это трёхстороннее сравнение, в котором более короткая строка всегда стоит раньше более длинной. Для проверки на *равенство* оно подходит, а для словарного порядка — плохо.

6.2 Перемещение и поиск

```
static inline int __util$movc5 (unsigned short *srclen, char *source, unsigned char *fill,
               unsigned short *dstlen, char *dest);
```

Команда MOVC5 процессора VAX в виде функции: переместить источник в назначение; если назначение длиннее, добить его значением `*fill`; если короче, лишние октеты источника не перемещаются; перекрытие безопасно. Длины передаются по адресу, как это делал VAX.

`char *__util$strstr(char *s1, size_t s1len, char *s2, size_t s2len)` ищет `s2` в `s1` при заданных обеих длинах и возвращает адрес совпадения или NULL. Она никогда не смотрит дальше `s1len`.

`__util$strcat_rx(dst, dstsz, &dstlen, src1, len1, src2, len2, ..., NULL)` склеивает список кусков, каждый из которых задан адресом и длиной, в один буфер и завершает список адресом NULL. Она возвращает `STS$K_SUCCESS`, когда всё поместилось, и `STS$K_ERROR`, если кусок не помещается (куски перед ним остаются, а результат завершается) или аргумент неверен. Один октет сохраняется под завершающий нуль, так что куски могут заполнить буфер ровно до `dstsz - 1` октетов, и в `dst[dstsz]` никогда ничего не пишется. Кусок нулевой длины пропускается.

`__util$fill(dst, dstsz, pattern, patternsz)` заполняет буфер, повторяя образец; последняя копия обрезается по размеру. Она возвращает `STS$K_ERROR` для адреса NULL, отрицательного размера или образца нулевой длины (которым нельзя заполнить ничего). `__util$strxor(key, keysz, src, srcsz, dst, dstsz, ctx)` применяет XOR к данным с повторяющимся ключом; если `ctx` не NULL, она запоминает позицию в ключе между вызовами. Это маска, а не шифр.

6.3 Очистка текста

Все три работают на месте и возвращают новую длину.

```
char    l_line [] = "  key = \"a ! b\" ! a comment  \n";
int l_len;

l_len = __util$uncomment(l_line, (int) strlen(l_line), '!');    /* cut at the first '!'
 ↪  outside quotes */
l_len = __util$trim(l_line, l_len);             /* drop blanks at both ends */
```

Рутина	Что делает
`__util$trim(s, len)`	Убирает ведущие и завершающие пробельные символы (`isspace`)
`__util$collapse(s, len)`	Убирает *все* пробельные символы. Результат завершается, если что-то было удалено; буфер из `len` октетов никогда не переполняется
`__util$uncomment(s, len, marker)`	Обрезает строку на первом октете `marker`, не находящемся в кавычках

Часть в двойных или одинарных кавычках пропускается как есть. Кавычка другого вида внутри неё — обычный октет (экранирования нет), а незакрытая кавычка тянется до конца строки. В примере `!` внутри кавычек уцелевает, а тот, что после них, начинает комментарий; результат — `key = "a ! b"`. Рутина возвращает 0, если там не было ничего, кроме комментария.

Это то, что нужно читателю файла настройки, и `__util$readconfig` построена из них.

6.4 Шаблоны

```
int __util$pattern_match (char *str, char *pattern);
```

Возвращает 1 при совпадении и 0 при его отсутствии (а также для аргумента NULL) — это не условное значение. `*` соответствует любой серии октетов (включая пустую), `%` — ровно одному, а `?` допускается как псевдоним `%`. Сравнение учитывает регистр, строка должна совпасть целиком, экранирования нет: буквальный `?` в шаблоне является подстановочным. Подстановочные знаки принадлежат только *шаблону*. `*` в проверяемой строке — обычный октет; `"*ba"` соответствует шаблону `"*a"`, как и `"?ba"` (до 1.7.0 это было не так).

```
__util$pattern_match("report.txt", "*.txt");    /* 1 */
__util$pattern_match("a1c", "a%c");     /* 1 */
__util$pattern_match("abc", "a?d");     /* 0 */
```

Сопоставитель итеративен — без рекурсии и без стека, который можно исчерпать, — поэтому время его работы ограничено произведением двух длин. Враждебный шаблон из множества звёздочек, которому когда-то требовалось 8,6 секунды, теперь занимает несколько сотен наносекунд. Шаблоны из файла настройки или от пользователя можно сопоставлять безопасно.

6.5 Base64

```
int __util$base64   (void *src, int srclen, void *dst, int dstsz, int *dstlen);
int __util$unbase64 (void *src, int srclen, void *dst, int dstsz, int *dstlen);
```

RFC 4648. Та же ограниченная последовательность вызова, что и у остального семейства: источник и длина, назначение и его размер, и полученная длина. `*dstlen` устанавливается даже тогда, когда буфер мал, так что вызывающий может узнать, какой размер ему нужен. Кодировщик дополняет `=` и добавляет завершающий нуль, если есть место. Декодер принимает ввод с дополнением и без него и отвергает всё прочее: октет вне алфавита, `=` в середине, висящий секстет. Он возвращает `STS$K_SUCCESS` или `STS$K_ERROR`.

```
char    l_out [32];
int l_len;

if ( 1 & __util$base64("Man", 3, l_out, sizeof(l_out), &l_len) )
    $LOG_FAO(STS$K_INFO, "encoded: !AD", l_len, l_out); /* TWFu */
```

6.6 Текст hex и UUID

Рутина	Что делает
`__util$bin2hex(src, dst, n)`	`n` октетов в `2n` шестнадцатеричных цифр строчными **и завершающий нуль** (нужно место на `2n + 1`); возвращает `2n`. Макрос `$BIN2HEX(s, d, l)` приводит типы
`__util$hex2bin(src, dst, n)`	`n` шестнадцатеричных цифр (в любом регистре) в `n/2` октетов; возвращает это число. Нечётное `n` или символ, не являющийся шестнадцатеричной цифрой, даёт **-1**, и ничего не записывается; `n` равное 0 даёт 0
`__util$uuid2str(uuid, buf, bufsz)`	16 октетов в 36-символьный текст строчными; возвращает длину
`__util$faohex(src, n, out, outsz, &outlen)`	Одна 16-октетная строка дампа (раздел 4.7); `outlen` может быть NULL

6.7 CRC-32

```
unsigned int __util$crc32c (unsigned int crc, const void *buf, size_t buflen);
```

Имя говорит о Кастаньоли, но рутина вычисляет **CRC-32 по IEEE 802.3** (тот, что в zlib, с отражённым полиномом `0xEDB88320`). Контрольное значение для `"123456789"` — `0xCBF43926`. Так сделано намеренно: имя уже использовалось программами, прежде чем кто-либо заметил, и теперь заголовок говорит об этом прямым текстом. Начинайте с `crc = 0`, а чтобы продолжить текущую сумму, передавайте предыдущий результат обратно как `crc`:

```
unsigned l_crc = 0;

l_crc = __util$crc32c(l_crc, l_block1, l_len1);
l_crc = __util$crc32c(l_crc, l_block2, l_len2);
```

Она возвращает саму контрольную сумму — эта рутина не использует соглашение об условных значениях. При первом вызове она выбирает свой движок: свёртку PCLMULQDQ на x86-64, если процессор её имеет, команды CRC32 на ARMv8, если об этом говорит `AT_HWCAP`, иначе таблицы slice-by-8. Все дают одни и те же биты. Переменная окружения `STARLET_CRC32=SOFTWARE` принуждает использовать таблицы, чем пользуется тест, чтобы проверить и их.

6.8 Списки элементов

Список элементов (item list) — это таблица записей (код, длина, адрес), которая говорит сервису, что извлечь или установить. Системные сервисы OpenVMS принимают их повсюду. В StarLet есть типы:

```
typedef struct __ile2 { unsigned short code, len; char *ptr; } ILE2;
typedef struct __ile3 { unsigned short code, len; char *ptr; unsigned short *retlen; }
 ↪  ILE3;
```

и `$ILE2INI(code, len, ptr)` как инициализатор, `$ILENULL` как ограничитель. Предостережение для ветерана: в ILE3 из OpenVMS первым идёт слово *длины*, а вторым *код*; здесь первым идёт код, а указатели имеют ширину указателя Linux. Это форма, а не двоичная раскладка, и список, построенный здесь, нельзя передавать системному сервису OpenVMS как есть. Заголовок говорит то же самое.

`$ILE3INI(item, code, len, ptr, retlen)` заполняет один *объект* ILE3 (не указатель на него) и является оператором, после которого нужна точка с запятой. Начиная с 1.7.0 он компилируется с обычными аргументами; прежде имена его параметров сталкивались с именами членов.

```
ILE3        l_items [3];
char        l_buf [32];
int     l_num = 0;
unsigned short  l_len = 0;

$ILE3INI(l_items[0], 1, sizeof(l_buf), l_buf, &l_len);
$ILE3INI(l_items[1], 2, sizeof(l_num), &l_num, NULL);
l_items[2] = (ILE3) { 0 };
```

6.9 Мелочи

- `$ARRSZ(a)` — число элементов массива.
- `$MIN(x, y)`, `$MAX(x, y)`, `$RANGE(x, lo, hi)`, `$ISINRANGE(x, lo, hi)` — над `int`. `$ISINRANGE2` — та же проверка; обе включают границы, в каком бы порядке они ни были заданы (`__util$isinrange2` до 1.7.0 границы исключала, вопреки собственному комментарию).
- `__util$iszero(buf, n)` — `STS$K_SUCCESS`, если все октеты нулевые, иначе `STS$K_WARN`. Буфер NULL или нулевая длина показать нечего и считаются «всё нуль».
- `__util$uint32_2_dec(value, dst, dstsz, &len)` — десятичный текст беззнакового значения, с ограничением.
- `__util$bzero(buf, n)` — очищает каждый из `n` октетов; `STS$K_ERROR` для буфера NULL с ненулевой длиной. (До 1.7.0 она оставляла нетронутыми последние от одного до трёх октетов.)
- `__util$mask2sym` и `__util$val2sym` учитывают разделитель и завершающий нуль, когда проверяют место, так что не поместившийся результат обрезается, но никогда не переполняет.

Глава 7. Ключевые слова и параметры

7.1 Поиск ключевого слова

У программы часто есть небольшой набор слов — глаголы консоли, названия режимов — и нужно значение, стоящее за словом, набранным пользователем. `__util$lookup_key` делает это по таблице `KWDENT`:

```
typedef struct _kwdent {
    ASC kwd;            /* the keyword, an ASCIC */
    union { int val; void *ptr; };  /* what it stands for */
} KWDENT;

/* an inline routine of utility_routines.h: */
int __util$lookup_key (const char *src, int srclen, KWDENT *tbl, int tblsz, KWDENT **kwd,
 ↪  int flags);
```

Флаг	Смысл
`UTIL$K_LOOKUP_NCASE`	Сравнивать без учёта регистра
`UTIL$K_LOOKUP_ABBREV`	Строка может быть начальной частью ключевого слова, как это принимает DCL

Без флагов строка должна совпадать с ключевым словом по длине и по каждому октету. Так всё устроено с 1.6.11, когда режим ABBREV исправили (он выходил за конец таблицы):

- **Полностью написанное ключевое слово побеждает**, чьим бы сокращением ни была ещё строка. `SET` — это `SET`, хотя и `SETUP` начинается с `SET`.
- **Сокращение выбирает ключевое слово, если выбирает только одно.** `SETU` — это `SETUP`.
- **Синонимы не двусмысленны.** Две записи с *одним и тем же значением* считаются одной. В таблице могут стоять `BOOT` и `B` с одним значением, и `bo` по-прежнему находит BOOT.
- **Неоднозначное сокращение — это `STS$K_WARN`.** `*kwd` устанавливается на *первое* подходящее ключевое слово, так что программа, желающая быть снисходительной, может им воспользоваться, а желающая быть строгой — откажет.
- **Ничего не найдено или пустая строка — это `STS$K_ERROR`**, а `*kwd` остаётся нетронутым.
- Возвращаемая запись всегда лежит внутри таблицы.

```
enum { K_BOOT = 1, K_SHOW, K_SET, K_SETUP };

static KWDENT   s_kwd [] = {
    { {$ASCINI("BOOT")},  {.val = K_BOOT} },
```

```
    { {$ASCINI("SHOW")},  {.val = K_SHOW} },
    { {$ASCINI("SET")},   {.val = K_SET} },
    { {$ASCINI("SETUP")}, {.val = K_SETUP} },
    };

KWDENT  *l_kwd = NULL;
int l_sts;

l_sts = __util$lookup_key("sho", 3, s_kwd, $ARRSZ(s_kwd), &l_kwd,
              UTIL$K_LOOKUP_NCASE | UTIL$K_LOOKUP_ABBREV);

if ( l_sts == STS$K_WARN )
    return  $LOG_FAO(STS$K_ERROR, "ambiguous keyword !AD", 3, "sho");
if ( !(1 & l_sts) )
    return  $LOG_FAO(STS$K_ERROR, "unknown keyword !AD", 3, "sho");
```

Здесь `sho` даёт `K_SHOW` со статусом `STS$K_SUCCESS`, а `se` даёт `STS$K_WARN`, и `*kwd` указывает на `SET`. Обратите внимание на проверку: WARN — *не* успех (младший бит сброшен), поэтому `1 & l_sts` его отвергает.

7.2 От значений к словам

Для журнала или экрана нужно обратное: слово для значения. Таблица `SYM_REC` ставит значению в соответствие его имя; она строится макросом `$SYM_REC_INI(NAME)`, который превращает имя в строку:

```
#define LNK$M_UP    1
#define LNK$M_ERR   2

static const SYM_REC    s_flags [] = {
    $SYM_REC_INI(LNK$M_UP), $SYM_REC_INI(LNK$M_ERR), $SYM_REC_EOL
    };

char    l_text [64];

__util$mask2sym(3, s_flags, l_text, sizeof(l_text));    /* "LNK$M_UP|LNK$M_ERR" */
__util$val2sym(2, s_flags, l_text, sizeof(l_text)); /* "LNK$M_ERR" */
```

`__util$mask2sym` соединяет через `|` имена всех установленных битов; `__util$val2sym` находит имя одного точного значения. Обе возвращают длину текста и завершают его. Если буфер мал, вывод останавливается раньше, а не переполняет его. Обе хороши для `!AD` в журнале.

7.3 Параметры командной строки и файла

Одна и та же таблица описывает параметры программы — для командной строки и для файла настройки. Одна запись `OPTS` называет параметр, говорит, где хранится его значение и какого оно типа:

```
typedef struct _opts {
    ASC name;       /* the option's name */
    void    *ptr;       /* where the value goes */
    size_t  sz;     /* the size of that place */
    int type;       /* OPTS$K_* */
} OPTS;
```

Тип	Переменная	Разбираемый текст
`OPTS$K_INT`	Целое ровно из `sz` октетов: 1, 2, 4 или 8 (0 означает `int`)	Число, десятичное, шестнадцатеричное с `0x` или восьмеричное с `0`, после него ничего нет. Размеры 1, 2, 8: **беззнаковые**, минус отвергается, от 0 до 2^n-1. Размеры 4 и 0: **знаковые**, минус принимается, диапазон int32
`OPTS$K_STR`	`ASC`	Любой текст, обрезанный по меньшему из `sz` и 254 октетов
`OPTS$K_PWD`	`ASC`	Как `STR`; `__util$showparams` скрывает его
`OPTS$K_OPT`	`int`	Переключатель: присутствие устанавливает 1
`OPTS$K_PORT`	`unsigned short`	Номер порта, от 0 до 65535, хранится в порядке байтов **узла**; чтобы положить в сеть, используйте `htons()`
`OPTS$K_IP4`	`struct in_addr`	Адрес IPv4 с точками, хранится так, как его делает `inet_aton`
`OPTS$K_CONF`	`ASC` или NULL	Имя файла настройки, читаемого сразу; внутри файла настройки не допускается

`OPTS$K_INT` любого другого размера — ошибка, и переменная остаётся нетронутой. (До 1.7.0 запись шла как в `int`, что переполняло `short` и было мало для `long`.) С 1.7.1 действует правило знака: параметр из 1, 2 или 8 октетов беззнаковый (`-1` — неверное значение, а не `0xFF`), а из 4 октетов или `int` — знаковый, от -2147483648 до 2147483647 (`0xFFFFFFFF` — неверное значение; 32-разрядную маску кладите в переменную из 8 октетов). `__util$showparams` печатает их так же.

Таблица заканчивается `OPTS_NULL`. Законченная программа:

```
#define __MODULE__  "OPTT"
#include    <stdio.h>
#include    <arpa/inet.h>
#include    "utility_routines.h"
```

```
#include    "starlet.h"

static  ASC     g_conf, g_name, g_pwd;
static  int     g_trace, g_count;
static  unsigned short  g_port = 4021;
static  struct in_addr  g_addr;
static  unsigned long long g_big;

static  const OPTS  s_opts [] = {
    { {$ASCINI("config")},   &g_conf,  sizeof(g_conf),  OPTS$K_CONF },
    { {$ASCINI("name")},     &g_name,  sizeof(g_name),  OPTS$K_STR },
    { {$ASCINI("password")}, &g_pwd,   sizeof(g_pwd),   OPTS$K_PWD },
    { {$ASCINI("trace")},    &g_trace, sizeof(g_trace), OPTS$K_OPT },
    { {$ASCINI("count")},    &g_count, sizeof(g_count), OPTS$K_INT },
    { {$ASCINI("big")},      &g_big,   sizeof(g_big),   OPTS$K_INT },
    { {$ASCINI("port")},     &g_port,  sizeof(g_port),  OPTS$K_PORT },
    { {$ASCINI("server")},   &g_addr,  sizeof(g_addr),  OPTS$K_IP4 },
    OPTS_NULL
    };

int main    (int argc, char *argv[])
{
    if ( !(1 & __util$getparams(argc, argv, s_opts)) )
        return  $LOG_FAO(STS$K_ERROR, "bad options");

    __util$showparams(s_opts);

    return  0;
}
```

Запустите с файлом `x.conf`, содержащим

```
! sample
-name=router7   ! the node
-cou=0x20
/server=10.1.2.3
-port=5000
-nosuch=1
```

как `./opts --config=x.conf --trace --password=sekrit -big=99999999999`; она напечатает (начиная с `%`, в сборке с `NDEBUG`; программа не определила `__FAC__`, поэтому в её записях стоит `UTILS`)

```
%UTILS-W: x.conf :6 : Skip unrecognized or unused option 'nosuch'
%UTILS-I: name[0:7] ='router7'
%UTILS-I: password[0:6] ='<password>'
%UTILS-I: trace = ON
%UTILS-I: count = 32 (0x00000020)
%UTILS-I: big = 99999999999 (0x000000174876E7FF)
%UTILS-I: port = 5000 (0x1388)
%UTILS-I: server = 10.1.2.3
```

Обратите внимание на `-cou` вместо `count`: в файле или в командной строке достаточно однозначного сокращения. Неизвестный `nosuch` — предупреждение в журнале, но статус вызова всё равно успех (`STS$K_INFO`, нечётный). Добавьте в файл строку `bogus`, не начинающуюся ни с `-`, ни с `/`, и вызов завершится неудачей:

```
%UTILS-E: x.conf :7 : Option must be started with a '-' or '/', skip : 'bogus'
%UTILS-E: bad options
```

Остальная часть файла всё равно читается.

Правила, как они есть в коде:

- **В командной строке** параметр начинается с `-`, `--` или `/`, а его значение идёт после `=`: `--port=5000`. Имя принимается в любом регистре и может быть сокращено до однозначной приставки: `--ser` вместо `server`. Имя, написанное полностью, побеждает, даже если оно же — начало более длинного (`name` и `nameserver`). Сокращение, подходящее двум параметрам (`--co` для `config` и `count`), — ошибка, записывается в журнал, и параметр пропускается. Имя *длиннее*, чем параметр, не совпадение, поэтому `NSERVERXYZ` не выбирает `NSERVER`.
- **Статус.** `__util$getparams` возвращает `STS$K_SUCCESS`, когда принят каждый аргумент. Она возвращает `STS$K_ERROR` при неверном значении, неизвестном или неоднозначном параметре, лексеме, не начинающейся с `-` или `/`, или сбое в файле настройки. Каждая ошибка записывается через `$LOG_FAO`, а остальные аргументы всё равно обрабатываются, так что один запуск сообщает обо всех. (До 1.7.0 вызов возвращал успех после всего этого.) Массив `argv` никогда не изменяется. Пустой аргумент пропускается.
- **В файле настройки** каждая строка имеет вид `-name=value` (или `/name=value`). `!` начинает комментарий, если он не стоит внутри двойных или одинарных кавычек. Пустые строки пропускаются. Действуют те же правила сокращений. Строка длиннее 254 октетов отбрасывается целиком с ошибкой (если то, что вышло за предел, не комментарий); её хвост не читается как отдельная строка. Параметр, которого нет в таблице, записывается в журнал как предупреждение и пропускается, потому что один файл может обслуживать несколько программ; он даёт статус `STS$K_INFO`, у которого младший бит установлен, так что `1 & status` по-прежнему говорит «успех». `OPTS$K_CONF` внутри файла отвергается с ошибкой: файл не называет другой файл. Когда `__util$readconfig` вызывается сама по себе, те же статусы составляют её возвращаемое значение; побеждает наихудший из всего файла (ошибка над информацией).
- **Кавычки.** Значение, начинающееся и кончающееся одной и той же кавычкой, `"a b"` или `'a b'`, теряет их — и в файле, и в командной строке (`/name="a b"`, когда оболочка пропустила кавычки). Одинокая или несбалансированная кавычка остаётся в значении. Правило действует для любого типа (`"42"` — число 42).
- **Переключатель** (`OPTS$K_OPT`) устанавливается в 1 своим присутствием. Выключить его из строки нельзя; дайте умолчанию такое значение, которое правильно без переключателя.
- **«Обязательного» нет.** Если параметр должен быть задан, программа сама проверяет переменную потом.
- **Строки идут в `ASC`**, так что значение обрезается на 254 октетах, а завершающий нуль всегда на месте.
- `__util$showparams` записывает всю таблицу в журнал с `STS$K_INFO`, значения в их типах, а для пароля показывает `<password>`.

`__util$readparams` годами была объявлена в заголовке и ни разу не определена. В 1.7.0 она убрана из заголовка; используйте `__util$getparams`.

Глава 8. Командная строка в стиле DCL

8.1 Замысел

В OpenVMS определение командного языка один раз говорило, как выглядит команда — её глагол, параметры, квалификаторы и тип значения каждого, — а разбор, проверку и справку делала система. Программа лишь спрашивала: *задан ли `/LOG`? каково значение P1?* `cli_routines` переносит эту идею на Си, заменяя файл CLD таблицами:

```
verb [sub-verb] [p1 p2 ... p8] [/qualifier[=value] | --qualifier[=value] ...]
```

Работа идёт от `argv`. Компилятора определений нет: таблицы пишутся как инициализаторы Си. Таблица глаголов — массив `CLI_VERB`, оканчивающийся записью с пустым именем; параметры и квалификаторы глагола — массивы `CLI_PQDESC`, оканчивающиеся так же.

8.2 Таблицы

```
typedef struct __cli_pqdesc__ {
    ASC     name;       /* the name */
    unsigned short  type;       /* CLI$K_FILE, CLI$K_NUM, ... */
    unsigned char   pn;     /* CLI$K_P1 ... CLI$K_P8 for a parameter, CLI$K_QUAL for a
     ↪  qualifier */
    unsigned char   flag;       /* CLI$M_NEGATABLE, CLI$M_LIST, CLI$M_REQUIRED */
    ASC     defval;     /* default value text, or empty */
    CLI_KEYWORD *kwd;       /* keywords, for CLI$K_KWD */
} CLI_PQDESC;

typedef struct __cli__verb__ {
    ASC     name;
    struct __cli__verb__ *next; /* sub-verbs, or NULL */
    CLI_PQDESC  *params, *quals;
    int     (*act_rtn) (struct __cli_ctx__ *clictx, void *act_arg);
    void        *act_arg;
} CLI_VERB;
```

Типы значений

Константа	Принимает
`CLI$K_FILE`	Любой текст; не проверяется
`CLI$K_NUM`	Беззнаковое число, десятичное, `0x` или восьмеричное; вся лексема должна состоять из цифр (без знака минус, без хвостового текста) и не переполнять
`CLI$K_DATE`	`dd-mm-yyyy`, за которым по желанию идут разделитель (`-`, пробел или `T`) и `hh:mm[:ss]`; день проверяется по месяцу, високосные годы учтены, и ничего не может следовать дальше
`CLI$K_IPV4`, `CLI$K_IPV6`	Адрес, который принимает `inet_pton`
`CLI$K_OPT`	Без значения: переключатель
`CLI$K_QSTRING`	Любой текст; не проверяется
`CLI$K_UUID`	Каноническая 36-символьная форма
`CLI$K_DEVICE`	Имя под `/dev`, которое должно существовать
`CLI$K_KWD`	Одно из слов `kwd` или однозначная приставка одного из них. Разбор сохраняет **полное** слово из таблицы, а не то, что набрано
`CLI$K_TIME`	Абсолютное время или дельта, проверяемые `__util$bintim` (глава 10)
`CLI$K_MAC`	Шесть шестнадцатеричных октетов, между которыми `:` или `-`

Флаги

Флаг	Смысл
`CLI$M_NEGATABLE`	У квалификатора есть форма `/NOname`
`CLI$M_LIST`	Он принимает список, `/tag=(a,b,c)` или повтор
`CLI$M_REQUIRED`	Он должен быть задан, иначе разбор не удаётся (относится и к параметрам)

Параметр необязателен, если он не `CLI$M_REQUIRED`.

8.3 Законченная программа

```
#define __MODULE__  "COPYT"
#define __FAC__     "COPYT"

#include    <stdlib.h>
#include    "utility_routines.h"
#include    "starlet.h"
#include    "cli_routines.h"

static  CLI_KEYWORD s_modes [] = {
    { .name = {$ASCINI("FAST")}, .val = 1 },
    { .name = {$ASCINI("SAFE")}, .val = 2 },
    { .name = {0} }
    };

static  CLI_PQDESC  s_params [] = {
    { .name = {$ASCINI("source")}, .type = CLI$K_FILE, .pn = CLI$K_P1, .flag =
    ↪  CLI$M_REQUIRED },
    { .name = {$ASCINI("target")}, .type = CLI$K_FILE, .pn = CLI$K_P2 },
    { .name = {0} }
    };

static  CLI_PQDESC  s_quals [] = {
    { .name = {$ASCINI("log")},    .type = CLI$K_OPT, .pn = CLI$K_QUAL, .flag =
    ↪  CLI$M_NEGATABLE },
    { .name = {$ASCINI("blocks")}, .type = CLI$K_NUM, .pn = CLI$K_QUAL, .defval =
    ↪  {$ASCINI("512")} },
    { .name = {$ASCINI("mode")},   .type = CLI$K_KWD, .pn = CLI$K_QUAL, .kwd = s_modes },
    { .name = {$ASCINI("tag")},    .type = CLI$K_QSTRING, .pn = CLI$K_QUAL, .flag =
    ↪  CLI$M_LIST },
    { .name = {0} }
    };

static  int s_do_copy   (CLI_CTX *a_ctx, void *a_arg)
{
ASC     l_val;
CLI_KEYWORD *l_kwd;
unsigned    l_state;

    if ( 1 & __cli$get_value(a_ctx, &s_params[0], &l_val) )
        $LOG_FAO(STS$K_INFO, "source = !AD", (unsigned) l_val.len, l_val.sts);

    if ( 1 & __cli$present(a_ctx, &s_params[1], NULL) )
        if ( 1 & __cli$get_value(a_ctx, &s_params[1], &l_val) )
            $LOG_FAO(STS$K_INFO, "target = !AD", (unsigned) l_val.len, l_val.sts);

    if ( 1 & __cli$get_value(a_ctx, &s_quals[1], &l_val) )      /* has a default */
        $LOG_FAO(STS$K_INFO, "blocks = !AD", (unsigned) l_val.len, l_val.sts);

    if ( 1 & __cli$present(a_ctx, &s_quals[0], &l_state) )
        $LOG_FAO(STS$K_INFO, "log is !AZ", (l_state & CLI$M_NEGATED) ? "negated" : "on");

    if ( 1 & __cli$get_keyword(a_ctx, &s_quals[2], NULL, &l_kwd) )
        $LOG_FAO(STS$K_INFO, "mode = !AC (value !SL)", &l_kwd->name, l_kwd->val);

    if ( 1 & __cli$present(a_ctx, &s_quals[3], NULL) )
        while ( 1 & __cli$get_value(a_ctx, &s_quals[3], &l_val) )
```

```
            $LOG_FAO(STS$K_INFO, "tag = !AD", (unsigned) l_val.len, l_val.sts);

    return  STS$K_SUCCESS;
}

static  CLI_VERB    s_verbs [] = {
    { .name = {$ASCINI("copy")}, .params = s_params, .quals = s_quals, .act_rtn =
     ↪  s_do_copy },
    { .name = {0} }
    };

int main    (int argc, char *argv[])
{
void    *l_ctx;
int l_sts;

    l_sts = __cli$parse(s_verbs, CLI$M_OPSIGNAL, argc - 1, argv + 1, &l_ctx);

    if ( l_sts == STS$K_WARN )          /* help was asked for, and has been printed */
        return  EXIT_SUCCESS;

    if ( 1 & l_sts )
        {
        l_sts = __cli$dispatch(l_ctx);
        __cli$cleanup(l_ctx);
        }

    return  (1 & l_sts) ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Несколько запусков:

```
$ copyt cop a.dat b.dat /nolog "--blocks=0x10" "/tag=(x,y)" /mode=sa
%COPYT-I: source = a.dat
%COPYT-I: target = b.dat
%COPYT-I: blocks = 0x10
%COPYT-I: log is negated
%COPYT-I: mode = SAFE (value 2)
%COPYT-I: tag = x
%COPYT-I: tag = y

$ copyt copy /blocks=12
%CLI_RTNS-F: Missing P1 - source !

$ copyt copy a.dat --blocks=12x
%CLI_RTNS-E: Value '12x' is not a valid number

$ copyt copy a.dat b c
%CLI_RTNS-F: Too many parameters, surplus token 'c'

$ copyt copy a.dat /blocks 5
%CLI_RTNS-F: Value '5' must follow '=' of the qualifier '/blocks' (write /blocks=5)

$ copyt copy a.dat --bogus
%CLI_RTNS-F: Unrecognized qualifier 'bogus'

$ copyt copy --help
Usage: copy source [target] [/qualifier ...]
Parameters:
```

```
    P1  source         FILE (./filename.ext, device:\path\filename.ext)
    P2  target         FILE (./filename.ext, device:\path\filename.ext)
Qualifiers:
    /log            OPTION (no value), /NO<name> accepted
    /blocks         DIGIT (decimal, octal, hex), default=512
    /mode           KEYWORD
    /tag            ASCII string in double quotes, list
```

(Собрано с `-DNDEBUG`; записи показаны начиная с `%`, без колонок даты, времени и потока. Обратите внимание, что для `copy` хватило `cop` и что `/mode=sa` сохранено как полное слово `SAFE`.)

8.4 Рутины

```
int  __cli$parse     (CLI_VERB *verbs, int opts, int argc, char **argv, void **clictx);
int  __cli$dispatch  (CLI_CTX *clictx);
int  __cli$get_value (CLI_CTX *clictx, CLI_PQDESC *pq, ASC *val);
int  __cli$present   (CLI_CTX *clictx, CLI_PQDESC *pq, unsigned *state);
int  __cli$cleanup   (CLI_CTX *clictx);
int  __cli$get_keyword (CLI_CTX *clictx, CLI_PQDESC *pq, const ASC *val, CLI_KEYWORD
 ↪  **kwd);
int  __cli$help      (CLI_VERB *verbs, CLI_VERB *verb);
void __cli$show_verbs(CLI_VERB *verbs, int level);
void __cli$show_ctx  (CLI_CTX *clictx);
```

`__cli$parse` сопоставляет `argv[0]` с таблицей глаголов (имя программы в неё *не* входит: передавайте `argv + 1`), затем подглагол, параметры и квалификаторы. Она строит контекст в `*clictx`, который необходимо освободить через `__cli$cleanup`, и возвращает условное значение. `opts` сочетает `CLI$M_OPSIGNAL` (записывать в журнал причину отказа) и `CLI$M_OPTRACE` (трассировка в отладочной сборке библиотеки). При неудачном разборе и после справки `*clictx` равен NULL, а наполовину построенный контекст освобождается, но описанная в документации идиома — всё равно вызвать для него `__cli$cleanup` — безопасна.

Результат — `STS$K_SUCCESS`; `STS$K_FATAL` для плохой команды (неизвестный или неоднозначный глагол или квалификатор, пропущенный обязательный элемент, лишнее слово) или `STS$K_ERROR` для значения не того типа; либо **`STS$K_WARN`, когда пользователь запросил справку**, она напечатана, и диспетчеризовать нечего. Справка — не отказ, и программа обязана так её и воспринимать:

```
if ( l_sts == STS$K_WARN )
    return  EXIT_SUCCESS;       /* the help was printed */
```

Справка распознаётся в любой лексеме: `/HELP`, `-HELP`, `--help`, `-h`, в любом регистре. Просто `help` — обычное слово: это может быть ваш собственный глагол или файл с таким именем. Если справка запрошена до глагола, печатается список команд; с глаголом — его параметры и квалификаторы с типами, умолчаниями и флагами; с глаголом-контейнером (`show --help`) — его подглаголы, а `show statistics --help` доходит до самого подглагола. Текст идёт через `__util$out`.

`__cli$get_value` возвращает текст параметра или квалификатора в `ASC`. Если элемент отсутствует и у него есть `defval`, вы получаете умолчание. Для квалификатора `CLI$M_LIST` каждый вызов возвращает следующий элемент, а вызов после последнего возвращает `STS$K_ERROR`. Для отсутствующего элемента без умолчания тоже возвращается `STS$K_ERROR`; элемент, присутствующий с пустым значением, даёт `STS$K_WARN`. Отсутствующий элемент — не ошибка программы, поэтому вызов **никогда не пишет о нём в журнал**, ни с `CLI$M_OPSIGNAL`, ни без него: зондирование бесплатно. (Контекст или дескриптор NULL — ошибка программирования, и о ней пишется в журнал.) Значение-ключевое слово возвращается как **полное слово таблицы**, какое бы сокращение ни набрали.

`__cli$get_keyword(clictx, pq, val, &kwd)` идёт на шаг дальше и отдаёт запись таблицы для элемента `CLI$K_KWD`: его полное имя в `kwd->name` и номер в `kwd->val`. Передайте значение, полученное от `__cli$get_value`, или NULL для первого значения элемента (или его умолчания). Курсор списка она не сдвигает. Она возвращает `STS$K_SUCCESS`; `STS$K_WARN`, если элемент присутствует с пустым значением (`/mode`, `/mode=`), как это говорит `__cli$get_value`; либо `STS$K_ERROR`, если у элемента нет таблицы ключевых слов, он отсутствует и не имеет умолчания либо значение не является словом таблицы; в журнал ничего не пишется. Использовать `__util$lookup_key` для сохранённого ключевого слова больше не нужно.

`__cli$present` говорит, был ли элемент задан: `STS$K_SUCCESS`, если так (`*state` несёт `CLI$M_PRESENT` и `CLI$M_NEGATED` для формы `/NO`), и `STS$K_WARN`, если нет. Если квалификатор повторён, решает *последнее* вхождение: `/LOG /NOLOG` — отрицание, а `/NOLOG /LOG` —

включено.

`__cli$dispatch` вызывает `act_rtn` самого глубокого сопоставленного глагола с контекстом и `act_arg` и возвращает то, что вернёт она. У рутины действия сигнатура `int (CLI_CTX *, void *)`.

8.5 Правила разборщика

- **Глагол или квалификатор можно сократить до однозначной приставки**, в любом регистре. Полностью написанное имя побеждает более длинное, началом которого оно служит: `/LOG` против `/LOGFILE` и, начиная с 1.7.0, `SET` против `SETUP`. Сокращение, подходящее двум глаголам (`se`), — ошибка. Глаголы с одним и тем же действием, параметрами, квалификаторами и подглаголами — синонимы и никогда не бывают неоднозначны друг с другом.
- **Лишние слова — ошибка.** Слово, которое не берёт ни одно место параметра, сообщается как `Too many parameters`, в том числе для глагола, у которого вообще нет параметров (раньше там оно игнорировалось). Глагол, которому нужен подглагол и который его не получил, говорит `Too few arguments`.
- **Квалификатор — это `/name` или `--name`.** Одиночный `-name` квалификатором не является. Значение следует за `=`: `/blocks=12`. Значение, отделённое пробелом (`/blocks 12`), не понимается; `12` будет лишней лексемой.
- **`/NOname` понимается** для квалификаторов, объявленных `CLI$M_NEGATABLE`. У отрицаемого квалификатора значения нет.
- **Списки:** `/tag=(a,b,c)` или `/tag=a,b,c`, для `CLI$M_LIST`.
- **Путь — это значение, но только там, где нужен путь.** Лексема, называющая определённый квалификатор, останавливает позиционный просмотр и оставляется квалификаторам. Лексема, начинающаяся с `/` и не называющая ни одного, принимается за значение ожидающего параметра, только если этот параметр `CLI$K_FILE` или `CLI$K_DEVICE` (`/etc/passwd` для `source`); для любого другого параметра или когда все места заполнены это ошибка `Unrecognized qualifier`. Неизвестный `--name` — всегда ошибка: `copy a.dat --bogus` не удаётся и больше не делает `--bogus` значением `target`.
- **`/blocks 5` — ошибка с сообщением.** Значение, идущее за квалификатором после пробела, не принимается как его значение, и разборщик говорит `Value '5' must follow '=' of the qualifier '/blocks'`. До 1.7.0 сообщение было просто `Unrecognized token`.
- **Значения проверяются строго.** `CLI$K_NUM` — беззнаковое число, каждый символ которого цифра и которое помещается; `CLI$K_DATE` проверяет день по месяцу. `-5`, `12x` и `31-02-2026` отвергаются.
- **Подглаголы** (`SHOW STATISTICS`): `next` глагола указывает на таблицу его подглаголов, которые следуют сразу за глаголом; квалификаторы идут после.
- **До восьми параметров**, от `CLI$K_P1` до `CLI$K_P8`.
- **Значения принимаются как строки**: тип — проверка, а не преобразование.
- `cli_routines.h` нуждается в `ASC`. Включайте `utility_routines.h` **до** него (`starlet.h` его подтягивает).

8.6 Чем это отличается от DCL

Здесь нет компилятора CLD и нет командных таблиц в файле. Нет запроса недостающего параметра, нет `/QUALIFIER`, у которого при наличии умолчание иное, чем при отсутствии, нет кода условия `CLI$_NEGATED` (`__cli$present` сообщает об отрицании в `*state`), нет `LIB$GET_INPUT`. Разборщик — та часть DCL, которая чаще всего нужна программе под Linux: глагол, параметры, квалификаторы и справка. Это он и делает.

Глава 9. Очереди и блокировки

9.1 Наследие

У VAX были команды для двусвязных очередей — INSQUE и REMQUE и их блокируемые родственники INSQHI, REMQTI и прочие. Очередь была повседневной структурой данных операционной системы, и программа для VAX опиралась на неё не задумываясь. StarLet даёт Linux очередь того же рода, хотя и не те же команды: двусвязный список с головой, хвостом, счётчиком и **спин-блокировкой**. Она не свободна от блокировок (lock-free) и не претендует на это.

9.2 Типы

```
typedef struct __entry {
    struct __entry  *left, *right;  /* neighbours; NULL at an end */
    void        *queue;     /* the queue the entry is in, or NULL */
} ENTRY;

typedef struct __queue {
    ENTRY   *head, *tail;
    int lock;           /* on Windows, an SRWLOCK */
    unsigned count;
} __QUEUE;

#define QUEUE_INITIALIZER { (ENTRY *) 0, (ENTRY *) 0, 0, 0 }
```

`ENTRY` должна быть **первым членом** того, что вы ставите в очередь:

```
typedef struct item {
    ENTRY   links;          /* first */
    int n;
} ITEM;
```

Поле `queue` элемента должно быть **нулевым** до его первой вставки. Статическая и обнулённая память такова; память из `malloc` — нет, если вы её не очистите.

9.3 Операции

Все возвращают условное значение, и все берут блокировку на время изменения.

Макрос	Что делает
`$INSQTAIL(que, ent, &count)`	Ставит элемент в хвост; в `count` помещается число элементов *до*
`$INSQHEAD(que, ent, &count)`	Ставит его в голову
`$REMQHEAD(que, &ent, &count)`	Берёт головной элемент в `ent`
`$REMQTAIL(que, &ent, &count)`	Берёт хвостовой элемент
`$REMQENT(que, ent, &count)`	Вынимает заданный элемент, где бы он ни был; в `count` — число до
`$CLRQUE(que, &count)`	Опустошает очередь, отсоединяя каждый элемент; в `count` — сколько их было
`$MOVQHEAD(que, ent)`, `$MOVQTAIL(que, ent)`	Перемещают элемент очереди в голову или в хвост; `ent` — сам элемент

```
__QUEUE l_que = QUEUE_INITIALIZER;
ITEM    l_item [3], *l_p;
unsigned l_count;

for ( int i = 0; i < 3; i++ )
    {
    l_item[i].n = i;
    l_item[i].links.queue = NULL;
    $INSQTAIL(&l_que, &l_item[i], &l_count);
    }

if ( 1 & $REMQHEAD(&l_que, &l_p, &l_count) && l_count )
    $LOG_FAO(STS$K_INFO, "took item !UL, there were !UL", l_p->n, l_count);
```

Что нужно знать; всё это проверено запуском (а с 1.7.0 — и `test_queue`, который к тому же прогоняет случайную последовательность всех операций против обычного массива):

- **Пустая очередь — это успех.** `$REMQHEAD` и `$REMQTAIL` на пустой очереди возвращают `STS$K_SUCCESS`, устанавливают `ent` в NULL, а `count` в нуль. Отличайте «получил элемент» от «там ничего нет» по `ent` (или `count`). До 1.7.0 `ent` оставался как был, а `$CLRQUE` оставлял `count` неустановленным. `$CLRQUE` на пустой очереди сообщает нуль.
- **`$REMQENT` элемента, которого нет в очереди,** — включая пустую очередь — это `STS$K_ERROR`: убирать нечего. Это отличается от изъятия с края, где пустая очередь — норма.
- **Элемент, уже находящийся в этой очереди,** — успех, он остаётся на месте; `count` не устанавливается. Элемент в *другой* очереди отвергается и `$INSQTAIL`, и `$INSQHEAD` с `UTIL$S_INQUE`; блокировка, которую не удалось взять, — `UTIL$S_NOLOCK`; аргумент NULL — `UTIL$S_INVARG`. (До 1.7.0 `$INSQHEAD` возвращал для всех трёх простой `STS$K_ERROR`.)
- **`$MOVQHEAD` и `$MOVQTAIL` работают.** Они переставляют голову, хвост и связи соседей; перемещение элемента, который уже на краю, — успех и ничего не меняет; элемент, которого нет в этой очереди, или аргумент NULL — `STS$K_ERROR`. Рутины принимают элемент как `void *` (они были объявлены с `void **ent`, что не согласовывалось с тем, что передают макросы).
- **Блокировка** — спин-блокировка. `$LOCK_LONG` (`__util$lockspin`) пытается около четырёх миллиардов раз и сдаётся с `STS$K_ERROR`, так что не зависает навсегда; `$UNLOCK_LONG` её освобождает. Она не рекурсивна. Используйте её для коротких критических участков.

9.4 Атомарные операции

- `$ATOMIC_INC(v)`, `$ATOMIC_DEC(v)` — `__sync_fetch_and_add` и `_sub` из GCC на единицу; они возвращают старое значение.
- `$CMP_STORE_LONG(addr, old, new)` — сравнение с обменом для `int`; истина, если запись выполнена. Дальний родственник пары `LDL_L`/`STL_C` из Alpha.

Это полные барьеры. Применяйте их для счётчика и флага; для чего-либо большего берите очередь или настоящую блокировку.

Глава 10. Время

10.1 Двоичное время здесь

В OpenVMS двоичное время — квадрослово, считающее тики по 100 наносекунд от 17-NOV-1858, а *дельта*-время — то же квадрослово, но отрицательное. StarLet сохраняет идею и отбрасывает единицу. Его двоичное время, `fao_time_t`, — это **секунды эпохи UNIX** (`time_t`), а дельта-время — **отрицательное** число секунд. Именно это читают директивы FAO `!%T` и `!%D`, так что форматировщик и преобразователи согласны между собой, и значение проходит путь туда и обратно.

Рутина	Что делает	По образцу
`__util$gettim(&t)`	Текущее время	`SYS$GETTIM`
`__util$bintim(text, &t)`	ASCII в двоичное, абсолютное или дельта	`SYS$BINTIM`
`__util$asctim(&len, &dsc, &t, flag)`	Двоичное в ASCII; `t` равное NULL означает «сейчас»	`SYS$ASCTIM`
`__util$add_times(&a, &b, &r)`	Сумма	`LIB$ADD_TIMES`
`__util$sub_times(&a, &b, &r)`	Разность	`LIB$SUB_TIMES`

Все возвращают условные значения. Сотые доли секунды принимаются, затем отбрасываются и печатаются как `.00`. Рутины отображения и директивы FAO одинаково читают знак числа: положительное — абсолютное время, отрицательное — дельта, а **нуль означает «сейчас»**, поэтому саму эпоху показать нельзя, а время до 1970 года вообще нельзя записать как абсолютное. Часовой пояс — константа `FAO$K_TZOFFSET`, нуль (UTC), если `starlet.c` не собран с другой; это пояс `bintim`, `asctim`, `!%D` и `!%T` в равной мере.

10.2 Текст

`__util$bintim` принимает, допуская пробелы вокруг:

Форма	Смысл
`dd-MMM-yyyy[ hh:mm[:ss[.cc]]]`	Абсолютное время; время суток по умолчанию 00:00:00
`hh:mm[:ss[.cc]]`	Сегодня в это время
`-MMM-` или `dd-MMM-`	Недостающие поля даты берутся от сегодняшнего дня
`dddd hh:mm[:ss[.cc]]` или `dddd-hh:mm...`	**Дельта**-время; результат отрицателен
`::10`	Дельта в десять секунд: поля времени можно опускать, как в DCL

Месяц — три буквы в любом регистре. День проверяется по месяцу, високосные годы учтены. Неверная или выходящая из диапазона строка даёт `FAO$K_IVTIME` (чётный, то есть неудача), а результат остаётся нетронутым. То же и для абсолютной даты до 01-JAN-1970: результат вышел бы отрицательным, и каждый потребитель прочёл бы его как дельту (до 1.7.0 так и происходило). Сама эпоха даёт 0. `__util$asctim` печатает абсолютное время как `dd-MMM-yyyy hh:mm:ss.00` с днём, дополненным пробелом, а дельту — как `dddd hh:mm:ss.00`; `flag = 1` просит только время суток.

10.3 Арифметика

Правила — как у `LIB$ADD_TIMES` и `LIB$SUB_TIMES`, при отрицательных дельтах:

Операция	Результат
дельта + дельта	дельта
абсолютное + дельта	абсолютное, более позднее
абсолютное + абсолютное	`FAO$K_IVTIME` (хотя бы одно должно быть дельтой)
абсолютное - абсолютное	дельта; `FAO$K_NEGTIM`, если первое раньше
абсолютное - дельта	абсолютное, более раннее; `FAO$K_NEGTIM`, если ушло бы ниже нуля
дельта - дельта	дельта; `FAO$K_NEGTIM`, если вторая больше
дельта - абсолютное	`FAO$K_IVTIME`

```
#define __MODULE__  "TIMEX"
#define __FAC__     "TIMEX"

#include    "utility_routines.h"
#include    "starlet.h"

int main    (void)
{
fao_time_t  l_t, l_d, l_r;
char        l_buf [64];
fao_desc_t  l_dsc = $FAO_DESC(l_buf);
unsigned short  l_len;

    __util$bintim("31-OCT-2026 12:30:00", &l_t);
    __util$bintim("1 02:00:00", &l_d);          /* one day and two hours */

    __util$add_times(&l_t, &l_d, &l_r);
    __util$asctim(&l_len, &l_dsc, &l_r, 0);         /*  1-NOV-2026 14:30:00.00 */
    $LOG_FAO(STS$K_INFO, "later: !AD", (unsigned) l_len, l_buf);

    __util$sub_times(&l_r, &l_t, &l_r);         /* absolute - absolute = delta */
    __util$asctim(&l_len, &l_dsc, &l_r, 0);         /* 1 02:00:00.00 */
    $LOG_FAO(STS$K_INFO, "difference: !AD", (unsigned) l_len, l_buf);

    return  0;
}
```

10.4 Остальные рутины времени

В заголовке `utility_routines.h` есть ещё три небольших помощника над `struct timespec`: `__util$add_time` (сумма с переносом наносекунд), `__util$sub_time(&t1, &t2, &result)` (разность `t1 - t2`) и `__util$cmp_time(&t1, &t2)`. Начиная с 1.7.0 разность нормализуется: `tv_nsec` всегда в пределах от 0 до 999999999, а отрицательная разность наносекунд занимает секунду (раньше оставался отрицательный `tv_nsec`). Сравнение возвращает знак `t1 - t2`: отрицательное, если `t1` раньше, нуль при равенстве, положительное, если позже, — на практике -1, 0 или 1, но проверяйте только знак. Оно сравнивает секунды, а не вычитает их, поэтому не может ни переполниться, ни перевернуть знак для далёких друг от друга времён; обе принимают указатели на `const`. И `__util$timbuf` раскладывает `time_t` в `struct tm` с уже скорректированными месяцем и годом (1-12, полный год). Префикс журнала ни одной из них не использует; у него собственная кэшированная выдача.

Глава 11. Протокол TLV

11.1 Замысел

`avproto` — способ класть типизированные значения в пакет и доставать их обратно, безопасно по порядку байтов. Пакет (PDU) — это заголовок и серия TLV: тег, длина, значение.

```
PDU:  | header 20 octets | TLV | TLV | ... |

header: | magic 8 | length 4 | csr 4 | seq 4 |            all in network byte order

TLV:    | type 4 bits + id 12 bits | length 16 bits | value 0..length octets |
```

- `magic` — сигнатура до восьми октетов, дополненная нулями; получатель её проверяет.
- `length` — число октетов TLV, следующих за заголовком.
- `csr` — слово команды или состояния, а `seq` — порядковый номер; оба в вашем распоряжении.
- **Тип** тега говорит, как читать значение: `TAG$K_BBLOCK` (0) октеты, `TAG$K_WORD` (3) 16 бит, `TAG$K_LONGWORD` (4) 32 бита, `TAG$K_QWORD` (5) 64 бита, `TAG$K_UUID` (8) 16 октетов. Слова преобразуются между сетевым порядком и порядком узла за вас.
- **Идентификатор** (id) ваш, от 0 до 4095.

Включайте `utility_routines.h` до `avproto.h`; ему нужны `STS$K_*`. Начиная с 1.7.0 `avproto.h` больше не отключает три предупреждения компилятора (`-Wparentheses`, `-Wdate-time`, `-Wunused-variable`) для каждого включающего его файла.

11.2 Рутины

```
inline static int __tlv$hset (void *pdu, const unsigned char *sig, unsigned csr, unsigned
 ↪  seq);
inline static int __tlv$hget (void *pdu, const unsigned char *sig, unsigned *len,
 ↪  unsigned *csr,
                  unsigned *seq);
int __tlv$put    (void *pdu, unsigned pdusz, unsigned tag, unsigned type, void *val,
          unsigned valsz);
int __tlv$get    (void *pdu, unsigned pdusz, int *context, unsigned tag, unsigned *type,
          void *val, unsigned *valsz);
int __tlv$lookup (void *pdu, unsigned pdusz, int *context, unsigned tag, unsigned *type,
          void **val, unsigned *valsz);
void __tlv$dump  (void *pdu, unsigned pdusz);
```

`__tlv$hset` инициализирует заголовок и устанавливает длину в нуль. Сигнатура — строка от 1 до 8 октетов, хранится дополненной нулями; более длинная или NULL — это `STS$K_ERROR`, а PDU не затрагивается (раньше её молча обрезали). `__tlv$hget` проверяет сигнатуру — по тому же правилу — и возвращает длину, `csr` и `seq` в порядке узла; при несовпадении она возвращает `STS$K_ERROR` и не записывает **ни одного** из трёх выходов (раньше она помещала `STS$K_ERROR` в `*csr`).

`__tlv$put` добавляет один TLV в буфер размера `pdusz` и обновляет длину в заголовке. Свободное место проверяется по реальной длине записи — 2, 4, 8 или 16 октетов для слова, длинного слова, квадрослова или UUID, `valsz` для блока, — поэтому за буфер она никогда не пишет, а PDU растёт ровно на запись плюс её четыре октета тега и длины. `valsz` для типа фиксированного размера должен быть *не меньше* размера типа (читается только столько октетов); нуль больше не принимается. (С X.94-06 размер был «необязательным» для этих типов, отчего нуль или малый размер и приводили к переполнению буфера.) Всё, что не подходит, — id тега выше 4095, тип выше 15, слишком малый буфер, испорченная длина заголовка, малый `valsz`, — это `STS$K_ERROR`, записывается в журнал, и ничего не пишется. Размер `pdusz` теперь `unsigned`, а не `unsigned short`, так что можно использовать буфер свыше 64 КБ.

`__tlv$get` находит тег и **копирует** значение в ваш буфер, преобразуя числа к порядку узла. Длина в линии для записи фиксированного размера должна равняться размеру её типа; запись из 3 октетов с тегом длинного слова — это `STS$K_ERROR`, как и UUID короче 16. `__tlv$lookup` возвращает **адрес** значения внутри PDU, без преобразования, что подходит для больших блоков октетов; длину записи фиксированного размера она проверяет так же, как `__tlv$get` (с 1.7.1; прежде она отдавала указатель на короткую запись), и неверная длина — это `STS$K_ERROR` без указателя. Обе проверяют каждую запись по реальному размеру `pdusz` полученного и не доверяют длине в заголовке: запись, выходящая за конец PDU, — это `STS$K_ERROR`. Обе возвращают `STS$K_WARN`, если тега нет. Для блока, UUID или неизвестного типа `valsz` обязателен: на входе — размер вашего буфера, на выходе — скопированные октеты.

`context` заставляет рутины итерировать: начните с `-1`; каждый вызов оставляет его после найденного TLV, так что второй вызов с тем же тегом находит следующий TLV с этим тегом.

```
#include    <stdio.h>
#include    "utility_routines.h"
#include    "avproto.h"

int main    (void)
{
unsigned char   l_pdu [128];
unsigned short  l_w = 0x1234;
unsigned    l_type, l_len, l_csr, l_seq, l_vlen;
int     l_ctx = -1;
```

```
char        l_buf [32];

    __tlv$hset(l_pdu, (const unsigned char *) "DEMOPDU", 1, 7);
    __tlv$put(l_pdu, sizeof(l_pdu), 1, TAG$K_WORD, &l_w, sizeof(l_w));
    __tlv$put(l_pdu, sizeof(l_pdu), 2, TAG$K_BBLOCK, "hello", 5);

    if ( 1 & __tlv$hget(l_pdu, (const unsigned char *) "DEMOPDU", &l_len, &l_csr, &l_seq)
     ↪  )
        {
        l_vlen = sizeof(l_buf);
        if ( 1 & __tlv$get(l_pdu, sizeof(l_pdu), &l_ctx, 2, &l_type, l_buf, &l_vlen) )
            printf("tag 2: type %u, %u octets: %.*s\n", l_type, l_vlen, (int) l_vlen,
 ↪  l_buf);
        }

    return  0;
}
```

(Эта программа использует `printf`, потому что это пример и тест; настоящий наш код использует `$LOG_FAO`.)

Она печатает `tag 2: type 0, 5 octets: hello`. PDU с этими двумя TLV имеет длину 20 + 6 + 9 = 35 октетов.

11.3 Предостережения

- Проверяйте статус `__tlv$put`. Запись, которая не помещается, id тега 4096 или больше и `valsz` меньше размера типа — всё это отвергается, и PDU остаётся как был.
- PDU в линии враждебен, пока не доказано обратное. Давайте `__tlv$get`, `__tlv$lookup` и `__tlv$dump` размер буфера, который вы *получили*, а не длину, которую заявляет заголовок.
- `__tlv$dump` пишет значение TLV строками по 64 октета, так что длинное значение больше не переполняет свой буфер (значение в 256 октетов и более писало на один октет дальше).
- Длину записи фиксированного размера проверяют и `__tlv$get`, и (с 1.7.1) `__tlv$lookup`.

Глава 12. Переносимость

12.1 Что где проверено

Платформа	Состояние
Linux x86-64 (glibc)	Всё, в CI и вручную; основная платформа
Linux с musl	CI собирает и запускает весь набор, в том числе со статической компоновкой, с `-Werror`
Linux ARMv8 (платы RK33xx/RK35xx)	Собрано и проверено; используется аппаратный CRC-32
Windows x64, MinGW-w64	Кросс-сборка из Linux; 22 из 24 тестовых программ собираются и запускаются под Wine (`log_prefix` только для Linux, а `logrotate` оставлен Windows 11, где он проходит)
Windows, MSYS2 с GCC 14	Собирается
Android (NDK, bionic)	Собирается с NDK; набором тестов на устройстве не проверялся

12.2 Правила, которым следует код

Ничего сверх libc. В Windows библиотека подключает `ws2_32` (для сокета syslog и значений адресов разборщика команд) и устанавливает `_WIN32_WINNT` в 0x0600 ради `WSASendMsg`.

Никакой жёсткой зависимости от дополнений glibc. `strings.h` включается явно ради `strncasecmp` (musl не экспортирует её из `string.h`). Номер потока берётся из `syscall(SYS_gettid)`, который предлагают glibc, musl и bionic; обёртка `gettid()` не используется, потому что bionic её скрывает.

Порядок байтов. `avproto` использует `be16toh` и родственные, а там, где в Windows нет `<endian.h>`, заголовок подставляет замены `_byteswap_*` (Windows здесь только little-endian).

Время. Префикс журнала читает `CLOCK_REALTIME_COARSE` в Linux. В Windows он читает `clock_gettime(CLOCK_REALTIME)` под MinGW-w64 и `timespec_get` иначе; доля секунды там, как и в Linux, — миллисекунды (до 1.7.0 Windows печатал шесть цифр).

Файлы журнала. В Windows файл журнала открывается с `FILE_SHARE_DELETE`, чтобы ротация могла переименовать его, пока он открыт; `ftruncate` — это `_chsize`.

CRC-32. Движок выбирается во время выполнения. На ARMv8 он запрашивает у `getauxval(AT_HWCAP)` `HWCAP_CRC32`, поэтому для использования аппаратного пути не нужен флаг `-march` (README всё ещё советует читателю его добавить; движки собраны с атрибутами цели, так что флаг не нужен). `STARLET_CRC32=SOFTWARE` принуждает использовать таблицы.

Android. Компилируйте с `-DANDROID_LOGCAT`, чтобы записи уходили ещё и в `logcat`.

Тип сборки. По умолчанию Release, с `NDEBUG`. Сборке для устройства без трассировки ничего больше не нужно.

Глава 13. Манера письма

Исходные тексты держатся манеры инженеров OpenVMS. Это не украшение: читатель, знающий соглашение, с первого взгляда видит время жизни и происхождение имени, а следующий читатель, то есть автор через год, скажет за это спасибо.

13.1 Имена

Форма	Значение
`g_name`	Глобальная
`l_name`	Локальная
`s_name`	Статическая: переменная или рутина, не экспортируемая из своего модуля
`a_name`	Параметр рутины
`__util$name`, `__cli$name`, `__tlv$name`	Экспортируемая рутина: префикс модуля, `$`, затем имя
`s_fao$name`	Статическая рутина: `s_`, модуль, `$`, имя
`FAC$K_NAME`	Константа; `FAC$M_NAME` — маска; `FAC$_NAME` — условное значение
`$NAME`	Макрос

`$` в имени — обычный Си для этих компиляторов, и по нему программист OpenVMS читает «это принадлежит средству, стоящему перед `$`».

13.2 Оформление

Стиль Whitesmiths: скобки вложенного блока стоят на отступе операторов, которые они охватывают, скобка рутины — в нулевой колонке. Табуляции шириной восемь колонок. Объявления в начале рутины, без отступа. Один параметр на строку в определении, закрывающая скобка на отдельной строке. Одиночный оператор под `if` обходится без скобок. Тип возврата рутины и её имя разделены табуляцией.

```
int __util$example  (
        int a_count
            )
{
int l_i;

    for ( l_i = 0; l_i < a_count; l_i++ )
        {
        if ( l_i == 3 )
            return  STS$K_SUCCESS;
        }

    return  STS$K_WARN;
}
```

13.3 Опознавание и правка

У каждого модуля два имени версии, `__IDENT__` и `__REV__`, определяемые в его начале и согласованные с историей изменений:

```
#define __IDENT__    "V.01-34"
#define __REV__      "1.34.0"
```

Правило: `X-aa-bbECOc` превращается в `aa.bb.c`. Функциональное изменение повышает `bb` и обнуляет ECO; небольшое исправление дефекта сохраняет `bb` и добавляет `ECOc`. Сейчас модули таковы:

Модуль	IDENT	REV
`utility_routines`	`V.01-34`	`1.34.0`
`starlet`	`X.01-16`	`1.16.0`
`cli_routines`	`X.00-17`	`0.17.0`
`avproto`	`X.94-12`	`94.12.0`

У заголовка `starlet.h` своё опознавание, `X.01-04`, правка `1.4.0`. У поставки в целом есть ещё одно число — версия в `project()` файла `CMakeLists.txt`, 1.7.1. Оно не зависит от опознаваний модулей и растёт с каждой поставкой, меняющей хоть что-то, так что по имени пакета ясно, какая это поставка.

13.4 История изменений и функциональное описание

Каждая запись истории содержит дату, инициалы автора (`RRL` — Ruslan R. Laishev, `CLD` — Claude), новое опознавание и то, что изменилось *и почему*. История — часть программы; это единственное место, где живут причины.

Каждая рутина открывается блоком в манере VMS:

```
/*
**++
**  FUNCTIONAL DESCRIPTION:
**
**  What the routine does, in plain sentences.
**
**  FORMAL PARAMETERS:
**
**  a_name  What it is, and whether it is input or output
**
**  IMPLICIT INPUTS/OUTPUTS:
**  What it reads or writes besides its parameters.
**
**  RETURN VALUE:
**  The condition values it can give, and what each means.
**--
*/
```

13.5 Условные значения, а не печать

Рутина возвращает условное значение и доставляет результат в выходном параметре. Сообщения и журнал идут **только** через `$LOG_FAO` (с явной серьёзностью `STS$K_*`) и `$PUTMSG_FAO` (с условным значением каталога); тонкая обёртка над ними допустима. Никаких `printf`, `fprintf`, `puts`, `perror`, `syslog` или голого `write(2)` для сообщения. Исключения — тесты и показ текста справки. Эта глава написана так, и так же написаны примеры этого руководства, кроме того, который говорит иное.

Глава 14. Тесты

14.1 Что есть

Запускайте все через `ctest --test-dir build` или, без CMake, через `sh tests/run.sh`. Каждый тест печатает собственный отчёт и завершается с ненулевым кодом при сбое. Каждый использует крошечную обвязку из `test_harness.h`: `CHECK(cond, message...)` и `TEST_DONE()`. На тесты домашнее правило о сообщениях не распространяется, и они могут печатать как угодно.

Тест	Что стережёт
`status_convention`	Младший бит: каждый код успеха нечётен, каждый код неудачи чётен; `bintim`, `asctim` и `CLI$K_TIME` соблюдают `1 & status`; `$FAC/$MSG/$SEV` обращают `$DEFFAC/$DEFMSG/$DEFSEV` поле за полем
`cli_features`	Умолчания, отрицаемые квалификаторы с правилом «побеждает последний», списки значений, `__cli$present`, обязательные элементы
`cli_help`	Встроенный распознаватель справки: `/HELP`, `-h`, `--help` — да, просто `help` — нет; `*clictx` обнуляется при раннем сбое
`cli_nested`	`SHOW STATISTICS`: подглагол ищется в списке родителя
`cli_filepath`	`/etc/passwd` остаётся значением, пока настоящий квалификатор всё ещё разбирается; одинокая лишняя лексема сообщается
`tlv`	Полный круг PDU: заголовок, слово, длинное слово, квадрослово, блоки, поиск, отсутствующий тег
`log_overflow`	Слишком большая запись усекается в границах, а её перевод строки или нуль попадает внутрь буфера; запуск под ASan
`setlogfd`	Журнал идёт в заданный дескриптор, не захватывая стандартные потоки; отрицательный дескриптор отвергается
`log_prefix`	Кэшированный префикс побайтно совпадает со старым `snprintf`; номер потока равен номеру ядра
`log_fao`	Обёртка `$LOG_FAO`, тело FAO, директивы адресов, форма без параметров
`logrotate`	Ниже предела — ничего; за ним `file.1 ... file.keep`; свежий файл для следующей записи; `keep=0`; keep свыше 99 отвергается; никакой ротации на `setlogfd`

Тест	Что стережёт
`putmsg_fao`	Сообщение каталога читает ровно те параметры, что ему даны; поиск, не зависящий от серьёзности; клеймо средства
`fao_addr`	`!%M`, `!%I`, `!%6`, `!%U`; угловые случаи RFC 5952; равенство с `inet_ntop()`
`pattern_match`	Фиксированные векторы, фаззинг на 500 000 случаев против эталона на динамическом программировании (с подстановочными символами и в проверяемых строках), псевдоним `?`, враждебный шаблон
`base64`	Векторы RFC 4648; круг по каждой длине; ввод без дополнения; отказы; короткий буфер всё же сообщает размер
`crc32`	Контрольные значения против побитового эталона; каждая длина до 2048 при каждом выравнивании; каждая точка разбиения; буфер 4 МБ; повторный запуск под `STARLET_CRC32=SOFTWARE`
`lookup_key`	Точный поиск, поиск без учёта регистра и по сокращению: полное ключевое слово побеждает, однозначное сокращение выбирает, синонимы не двусмысленны, неоднозначное — `STS$K_WARN`, запись лежит внутри таблицы
`fao_edge`	Управляющая строка, оборванная в каждой директиве, в блоках кучи точного размера под ASan; год выше 9999; поле свыше 512 октетов; короткий список параметров; коды `FAO$K_*`; нулевое и дельта-время; `starlet.h` как первое включение
`tlv_hostile`	Попытки переполнения `__tlv$put` для каждого типа, усечённые и лгущие записи и неверные длины для `get` и `lookup`, короткий UUID, дамп в 300 октетов, сигнатура свыше восьми октетов, длина заголовка против записей
`cli_rules`	`SET`/`SETUP`, синонимы, лишние слова, справка подглагола, сохранённое ключевое слово и `__cli$get_keyword` (в том числе пустое значение), неизвестные квалификаторы, молчаливое зондирование, `/q value`, проверки даты и числа
`queue`	Правило пустой очереди, коды состояния, `remqent` чужака, перемещения первого, последнего и среднего элемента и случайное блуждание всех рутин против обычного массива
`strings`	`hex2bin`, `bzero`, `fill`, `movc5`, `strelem`, `str2asc`, `strcat_rx`, `collapse`, `uncomment`, `mask2sym`/`val2sym`, `isinrange2`, помощники `timespec`, `$ILE3INI` и контрольное значение CRC-32; буферы точного размера

Тест	Что стережёт
`params`	Размеры и знаковость `OPTS`, кавычки, сокращения, статус после ошибок, нетронутый `argv`, тексты в 255 октетов, файл настройки, `showparams`
`log_misc`	Миллисекундная доля, серьёзность и syslog рутин журнала, форма `[mod\func:line]`, метка `[FAO ...]`, неопределённая серьёзность, `faohex` с NULL вместо длины, статусы `inimsg`, в том числе из многих потоков, `syslog` без хоста

Это двадцать четыре программы и двадцать пять прогонов. Семь новых тестов версии 1.7.0 — те, которых не хватало, когда писалось первое издание этого руководства: очереди, `__util$bzero` и её соседи, разборщик параметров и углы FAO, командной строки и рутин TLV. Не проверяется вовсе: сырой писатель `__util$out` сам по себе (он задействован через текст справки и тесты журнала). `tests/run.sh` теперь перечисляет все программы; это тот же набор, что у CTest, за вычетом второго прогона `crc32`.

Приложение A. Краткая справочная карта

Порядок включения: `utility_routines.h` (или `starlet.h`, который его подтягивает), затем `cli_routines.h` или `avproto.h`, которым он нужен первым. Определите `__MODULE__` и `__FAC__` перед всеми ними.

Условные значения

Макрос	Значение
`1 & sts`	Проверка: успех, если не нуль
`$DEFFAC(n) $DEFMSG(n) $DEFSEV(s)`	Построить условное значение из частей
`$FAC(c) $MSG(c) $SEV(c)`	Разобрать условное значение на части
`$STS_ERROR(c)`	Истинно для ERROR и FATAL
`STS$K_WARN 0, SUCCESS 1, ERROR 2, INFO 3, FATAL 4`	Серьёзности
`STS$M_SYSLOG`	Добавить по OR к серьёзности: ещё и в `syslog(3)`, `LOG_INFO` или `LOG_ERR`
`FAO$K_NEGTIM 0x08, BUFOVF 0x0A, IVTIME 0x0C, BADDIR 0x12, NOPRM 0x1A, FLDOVF 0x22`	Коды завершения FAO и времени
`UTIL$S_INVARG 0x102, INQUE 0x202, NOLOCK 0x302, DUPFAC 0x400`	Коды очередей и `__util$inimsg`

Журнал и сообщения

Макрос	Значение
`$LOG_FAO(sev, ctl, ...)`	Запись журнала; управляющая строка FAO; не более 16 параметров; возвращает серьёзность
`$LOG(sev, fmt, ...)`	Запись журнала; формат `printf`; возвращает серьёзность
`$PUTMSG_FAO(sts, ...)`	Сообщение каталога; возвращает `sts`
`$TRACE(fmt, ...)`, `$IFTRACE(cond, fmt, ...)`	Отладочная трассировка; с `NDEBUG` — ничего
`$DUMPHEX(ptr, len)`	Дамп hex в журнал
`__util$inimsg(&desc)`	Зарегистрировать каталог (массив не должен быть `const`); при отказе `UTIL$S_INVARG`, `UTIL$S_DUPFAC`
`__util$deflog(file, NULL)`, `__util$setlogfd(fd)`	Выбрать назначение журнала
`__util$rotatelogfile(limit, keep)`	Ротировать файл журнала
`__util$syslog(fac, sev, tag, msg, len)`	Одна датаграмма UDP сборщику, запомненному `__util$deflog` (больше никто ему не шлёт)

FAO: директивы приведены в разделе 5.3; `$FAO_DESC(buf)` — инициализатор дескриптора.

Строки и данные

Макрос	Значение
`$ASCDECL(a, "s")`, `$ASCINI("s")`, `$ASCLEN(p)`, `$ASCPTR(p)`, `$ASC(p)`	Счётные строки
`$ARRSZ(a)`, `$MIN`, `$MAX`, `$RANGE`, `$ISINRANGE`	Мелкие помощники
`$BIN2HEX(s, d, n)`	Октеты в текст hex
`$SYM_REC_INI(NAME)`, `$SYM_REC_EOL`	Строки таблицы «значение — имя»
`$ILE2INI(code, len, ptr)`, `$ILENULL`	Инициализатор элемента списка элементов, ограничитель
`$ILE3INI(item, code, len, ptr, retlen)`	Заполняет объект ILE3; оператор

Очереди и блокировки

Макрос	Значение
`QUEUE_INITIALIZER`	Инициализатор `__QUEUE`
`$INSQTAIL $INSQHEAD $REMQHEAD $REMQTAIL $REMQENT $CLRQUE`	Операции; каждая возвращает условное значение. Пустая очередь даёт SUCCESS, `ent` равен NULL, `count` равен 0
`$MOVQHEAD(que, ent)`, `$MOVQTAIL(que, ent)`	Переместить элемент очереди на край
`$LOCK_LONG(p)`, `$UNLOCK_LONG(p)`	Спин-блокировка на `int`
`$ATOMIC_INC(v)`, `$ATOMIC_DEC(v)`, `$CMP_STORE_LONG(p, old, new)`	Атомарные операции

Командная строка

Макрос	Значение
`CLI$K_P1 ... CLI$K_P8`, `CLI$K_QUAL`	Позиция элемента
`CLI$K_FILE NUM DATE IPV4 IPV6 OPT QSTRING UUID DEVICE KWD TIME MAC`	Типы значений
`CLI$M_NEGATABLE`, `CLI$M_LIST`, `CLI$M_REQUIRED`	Флаги дескриптора
`CLI$M_PRESENT`, `CLI$M_NEGATED`	Биты состояния от `__cli$present`
`CLI$M_OPSIGNAL`, `CLI$M_OPTRACE`	Параметры `__cli$parse`
`__cli$get_keyword(ctx, pq, val, &kwd)`	Запись таблицы для сохранённого значения `CLI$K_KWD` (новое в 1.7.0)
Результат `__cli$parse`	SUCCESS; WARN = справка напечатана, не диспетчеризовать; FATAL = плохая команда; ERROR = плохое значение

Ключевые слова, параметры, TLV: `UTIL$K_LOOKUP_NCASE`, `UTIL$K_LOOKUP_ABBREV`; `OPTS$K_INT STR OPT PORT IP4 CONF PWD` и `OPTS_NULL`; `TAG$K_BBLOCK 0, WORD 3, LONGWORD 4, QWORD 5, UUID 8`.

Приложение B. История библиотеки

Взято из истории изменений `CMakeLists.txt`. Номер — это версия в `project()`, которая повышается с каждой поставкой, меняющей модуль.

Дата	Версия	Что изменилось
08-MAR-2024	-	Начат набор, как набор «служебных рутин для программистов на чистом Си»
24-MAR-2026	-	Добавлен `__MAIN_FOR_DEBUG__` для отладки и разработки
01-AUG-2026	-	Правила установки и упаковки действуют только для сборки верхнего уровня; набор устанавливается как пакет CMake с `StarLet::starlet`; CPack
07-AUG-2026	1.1.0	Версия пакета становится единым номером поставки
07-AUG-2026	1.1.1	Обновлены руководства: `PUTMSG_FAO`, синтаксис команд, `--name`, ошибка нераспознанной лексемы
07-AUG-2026	1.2.0	Поиск сообщения стал независим от серьёзности
07-AUG-2026	1.2.1	Поиск сравнивает средство, затем номер сообщения; найденная запись принимает серьёзность вызывающего
07-AUG-2026	1.2.2	Имя средства возвращается в дескриптор записей; обёртка `%facility-S-text`
07-AUG-2026	1.3.0	Директивы адресов FAO `!%M`, `!%I`, `!%6`, `!%U`
13-AUG-2026	1.4.0	`!%6` даёт каноническую форму RFC 5952; в руководствах появилось приложение по FAO

Дата	Версия	Что изменилось
13-AUG-2026	1.4.1	Кэшированный префикс записи журнала, запись примерно вчетверо быстрее
13-AUG-2026	1.4.2	`!%6` показывает адрес, отображённый из IPv4, в смешанной записи
14-AUG-2026	1.4.3	`project()` говорит `LANGUAGES C`; компилятор C++ не нужен
14-AUG-2026	1.4.4	Записано правило IDENT/REV
14-AUG-2026	1.4.5	Домашние правила по всему пакету; архив называется `<package>-<ident>.tar.gz`
14-AUG-2026	1.4.6	`pattern_match` стал итеративным; враждебный шаблон упал с 8,6 с до 318 нс
14-AUG-2026	1.4.7	Задача musl в CI
14-AUG-2026	1.4.8	Восемь рутин проверены по шаблону рутины
29-AUG-2026	1.5.0	`$LOG_FAO`
29-AUG-2026	1.5.1	Поставка распаковывается в `starlet-<ident>/`
31-AUG-2026	1.5.2	Имя средства проставляется в каждую запись сообщения при `inimsg`
01-SEP-2026	1.5.3	Никаких идентификаторов со знаком доллара в файлах CMake
01-SEP-2026	1.6.0	`?` как псевдоним `%` в `pattern_match`
01-SEP-2026	1.6.1	Документация приведена в порядок
10-SEP-2026	1.6.2	`base64` и `unbase64`
11-SEP-2026	1.6.3	Исправлен `$MSG()`, чтобы он обращал `$DEFMSG()`
19-SEP-2026	1.6.4	Поле мнемоники в записи сообщения; `%FAC-S-MNEMONIC`
03-OCT-2026	1.6.5	CRC-32 выбирает движок во время выполнения (PCLMULQDQ, ARMv8 CRC32, таблицы)
03-OCT-2026	1.6.6	Release — тип сборки по умолчанию; `NDEBUG` идёт с ним
04-OCT-2026	1.6.7	`__util$rotatelogfile`
06-OCT-2026	1.6.8	Номер потока через `syscall(SYS_gettid)`; сборки для Android NDK
20-OCT-2026	1.6.9	Собирается GCC 14 в Windows (MSYS2)

Дата	Версия	Что изменилось
22-OCT-2026	1.6.10	Windows x64 с MinGW-w64 из Linux; тесты под Wine; журнал разделяет удаление, поэтому ротация там работает
31-OCT-2026	1.6.11	Исправлен режим ABBREV в `__util$lookup_key`; `test_lookup_key`; первое издание Руководства программиста и Справочного руководства (номера изданий -0111), написанных по этой поставке, со списком найденных ими дефектов
31-OCT-2026	1.7.0	Исправлены дефекты 1.6.11: `utility_routines` V.01-34 (очереди, строки и буферы, параметры и файлы настройки, журнал и syslog, потокобезопасный `inimsg`, новый сопоставитель шаблонов), `starlet` X.01-16 (ограниченный сканер FAO, условные значения `FAO$K_*`, правила времени), `cli_routines` X.00-17 (сопоставление глаголов как в DCL, строгие значения, `__cli$get_keyword`), `avproto` X.94-12 (рутины TLV, безопасные по границам), `starlet.h` X.01-04 (включает `utility_routines.h`); семь новых тестов; оба руководства пересмотрены (-0170); приложение D перечисляет изменения

Дата	Версия	Что изменилось
31-OCT-2026	1.7.1	Восемь пунктов, оставшихся от 1.7.0: `__tlv$lookup` проверяет длину на линии; рутины журнала возвращают `STS$K_ERROR` для неопределённой серьёзности; метка `[FAO ...]` для неверной управляющей строки; `__util$hex2bin` проверяет ввод; `OPTS$K_INT` 1/2/8 беззнаковые, 4/0 знаковые; кавычки снимаются со значения; `__cli$get_keyword` даёт WARN для пустого значения; комментарий `__util$deflog`; `avproto` X.94-12ECO1, `utility_routines` V.01-34ECO1, `cli_routines` X.00-17ECO1; оба руководства пересмотрены (-0171)

Собственные истории модулей уходят глубже: `utility_routines.h` — в сентябрь 2017 года, рутины журнала и `$GETMSG` — в 2017 и 2021 годы, `cli_routines` — в август 2018 года, `avproto` — в июль 2018 года.

Приложение C. Известные ограничения: что остаётся в силе

Первое издание этого руководства несло длинный список дефектов, найденных в исходных текстах 1.6.11. В 1.7.0 они исправлены, а приложение D перечисляет, что изменилось. Ниже — то, что остаётся верным для библиотеки, найденное чтением исходных текстов и запуском небольших программ. Большая часть из этого — библиотека такой, какой её задумали; остальное — углы. Это руководство ничего не скрывает.

По замыслу

- `__util$crc32c` вычисляет **CRC-32 по IEEE 802.3** (контрольное значение `0xCBF43926` для `"123456789"`), а не CRC-32C Кастаньоли, как намекает её имя. Имя сохранено, потому что от него зависят программы, а заголовок теперь говорит, что это за рутина.
- Двоичное время **нуль означает «сейчас»** в `!%D`, `!%T` и `__util$asctim`, а **отрицательное время — это дельта**. Дату до 1970 года нельзя записать как абсолютное время; `__util$bintim` отвергает такую.
- Значение квалификатора должно следовать за `=`: `/blocks=5`. Форма `/blocks 5` отвергается, теперь с сообщением, объясняющим почему.
- **Раскладка ILE3 ставит код первым**, затем длину; в раскладке OpenVMS первой идёт длина. Структура — форма для переносимого кода, а не двоичный формат для системного сервиса.
- `__util$fao`, форма с переменным числом аргументов, не может знать, сколько параметров ей дали. Используйте макросы или `__util$faol`, у которой есть счёт.
- **Движок FAO ничего не пишет в журнал** и сообщает о неверной управляющей строке только через статус. `$LOG_FAO` и `$PUTMSG_FAO` добавляют к записи метку `[FAO ...]` (с 1.7.1), но возвращаемое значение сохраняют; программе они сообщить не могут. Проверьте сообщение один раз, посмотрев на него.
- Серьёзность выше `STS$K_FATAL` печатается как `?`; с 1.7.1 рутины журнала возвращают для неё `STS$K_ERROR`. Передавайте только пять серьёзностей.
- `__util$cmpasc` для двух счётных строк ставит более короткую строку перед более длинной, каковы бы ни были буквы: это проверка на равенство, а не словарный порядок.
- Элемент, уже находящийся в очереди, — успех для `$INSQTAIL` и `$INSQHEAD`, а `count` не устанавливается. Проверяйте `count` только после изъятия.
- У шаблона нет экранирования: `*`, `%` и `?` нельзя сопоставить буквально.
- Переключатель (`OPTS$K_OPT`) можно только включить из командной строки, а библиотека не может сообщить программе, что параметр обязателен. `ASC`, содержащая текстовое значение, обрезается на 254 октетах.
- Часть записи журнала `[module\routine:line]` появляется только в единице, скомпилированной без `NDEBUG`. После `fork()` потомок продолжает нести в префиксе журнала номер потока своего родителя до exec.
- `$LOG_FAO` и `$PUTMSG_FAO` принимают не более шестнадцати параметров, а тело

записи обрезается на 2047 октетах.
- В библиотеке нет векторов сообщений, нет обработчиков условий, нет `LIB$GET_INPUT`, нет компилятора CLD, нет `__util$traceback` и нет очереди без блокировок.

Углы, которые не по замыслу

- README всё ещё советует пользователю ARMv8 добавить `-march=armv8-a+crc` ради аппаратного CRC-32. Движок выбирается во время выполнения и флага не требует.
- `__cli$show_ctx` и `__cli$show_verbs` — отладочные средства; их вывод идёт в журнал, а его раскладка не обещана.

Приложение D. Изменения в версиях 1.7.0 и 1.7.1

Изменения в версии 1.7.1

Версия 1.7.1 исправляет восемь пунктов, которые остались открытыми при написании руководств 1.7.0. Опознавания модулей: `avproto` X.94-12ECO1, `utility_routines` V.01-34ECO1 и `cli_routines` X.00-17ECO1; версия пакета — 1.7.1. Изменения, способные заставить существующую программу вести себя иначе, помечены **(проверьте)**.

- **`__tlv$lookup` проверяет длину на линии** записи фиксированного размера (слово 2, длинное слово 4, счетверённое слово 8, UUID 16), как `__tlv$get`. Неверная длина — `STS$K_ERROR`, указатель не выдаётся. Прежде указатель на короткую запись возвращался как успех **(проверьте)**.
- **Неопределённая серьёзность возвращает `STS$K_ERROR`.** `$LOG`, `$LOG_FAO`, `__util$log`, `__util$log_fao`, `__util$logd`, `__util$logd_fao` и `__util$log2buf` по-прежнему печатают `?` для серьёзности 5, 6 или 7, но возвращают `STS$K_ERROR` (2, младший бит сброшен) вместо `STS$K_UNDEF` (5, младший бит установлен), так что `return $LOG(sev, ...)` с неверной серьёзностью больше не сообщает об успехе **(проверьте)**. Внутри StarLet ничто на 5 не опиралось. Значения 0–4, с `STS$M_SYSLOG` или без, не изменились.
- **Неверный статус FAO стал виден.** Когда форматировщик `$LOG_FAO`, `$PUTMSG_FAO` или их отладочных форм возвращает статус со сброшенным младшим битом, запись заканчивается меткой `[FAO BADDIR]`, `[FAO NOPRM]`, `[FAO FLDOVF]` или `[FAO BUFOVF]` (`[FAO 0x...]` для другого статуса); запись, заполнившая буфер, обрезается, чтобы освободить место метке. Запись пишется, возвращаемое значение прежнее; программа, разбирающая журнал, увидит метку **(проверьте)**.
- **`__util$hex2bin` проверяет ввод.** Нечётная длина или символ, не являющийся шестнадцатеричной цифрой, даёт **`-1`**, и ничего не записывается; текст проверяется до того, как сохранён первый октет. Прежде нечётная длина давала первый октет из одной цифры, а неверный символ — бессмысленное значение **(проверьте)**. Длина 0 по-прежнему даёт 0. В дереве исходных текстов нечётную длину никто не применял.
- **У `OPTS$K_INT` есть правило знака.** Размеры 1, 2 и 8 беззнаковые: ведущий `-` отвергается (раньше он сохранялся как `0xFF`, `0xFFFE`, …), диапазон от 0 до 2^n-1. Размеры 4 и 0 (`int`) знаковые, диапазон от -2147483648 до 2147483647: значение от 2147483648 до 4294967295 или `0xFFFFFFFF` теперь отвергается (раньше сохранялось как отрицательное число) **(проверьте)**. Параметру с 32-разрядной маской дайте размер 8. `__util$showparams` печатает 1, 2 и 8 как беззнаковые, 4 и 0 как знаковые. Комментарий к `OPTS$K_INT` в заголовке это говорит.
- **Значение в кавычках теряет кавычки.** Если значение после отсечения пробелов начинается и кончается одной и той же кавычкой, `"` или `'`, — в файле настройки или в командной строке, — оно сохраняется без них **(проверьте)**. Одинокая или несбалансированная кавычка остаётся. Правило действует для любого типа параметра; программа, снимавшая кавычки сама, снимать будет нечего.
- **`__cli$get_keyword` возвращает `STS$K_WARN`** для элемента, присутствующего с

пустым значением (`/mode`, `/mode=`), как `__cli$get_value`. Отсутствие по-прежнему `STS$K_ERROR`. Программа, проверявшая `1 & status`, не затронута; сравнивавшая с `STS$K_ERROR` — затронута **(проверьте)**.

- **Комментарий `__util$deflog` исправлен**: она запоминает хост syslog, ничего ему не отправляется, если программа не вызывает `__util$syslog`; `STS$M_SYSLOG` идёт в локальный `syslog(3)`.
- Тесты: `tlv_hostile` (lookup, 9 проверок), `log_misc` (8), `strings` (5), `params` (8) и `cli_rules` (8) получили проверки.

Изменения в версии 1.7.0

Версия 1.7.0 исправляет дефекты, найденные при написании первого издания руководств для 1.6.11, и ужесточает несколько правил, бывших свободными. Это приложение перечисляет каждое изменение, которое могла бы заметить использующая библиотеку программа. Опознавания модулей: `utility_routines` V.01-34, `starlet` X.01-16, `cli_routines` X.00-17, `avproto` X.94-12 и заголовок `starlet.h` X.01-04.

Изменения, способные заставить существующую программу вести себя иначе, помечены **(проверьте)**. Посмотрите на них в первую очередь.

Условные значения

- Коды FAO и времени теперь настоящие условные значения, все чётные: `FAO$K_BUFOVF` 0x0A, `FAO$K_IVTIME` 0x0C и `FAO$K_NEGTIM` 0x08 как прежде, **`FAO$K_BADDIR` 0x12** (был 6) **(проверьте)**. Новые: `FAO$K_NOPRM` 0x1A (слишком мало параметров) и `FAO$K_FLDOVF` 0x22 (текст поля свыше 512 октетов, обрезан).
- `UTIL$S_NOLOCK` равен **0x302** (был 0x202, как у `UTIL$S_INQUE`) **(проверьте)**. `UTIL$S_DUPFAC` 0x400 новый. Все четыре кода `UTIL$S_*` объявлены в одном месте, со сброшенным младшим битом.
- `$INSQHEAD` возвращает те же коды, что и `$INSQTAIL`: `UTIL$S_INVARG`, `UTIL$S_INQUE`, `UTIL$S_NOLOCK`. Раньше для всех них он возвращал простой `STS$K_ERROR`.
- `__util$inimsg` возвращает `UTIL$S_INVARG` для дескриптора NULL и `UTIL$S_DUPFAC` для уже зарегистрированного средства; оба раньше были `STS$K_WARN`. Она потокобезопасна. Строка, печатаемая для неизвестного условия, начинается с `%`.
- Когда вызов FAO находит более одной ошибки, его статус — первая из них.

Включения и объявления

- `starlet.h` сам включает `utility_routines.h` и больше не определяет запасные значения серьёзностей: два заголовка можно включать в любом порядке. `cli_routines.h` и `avproto.h` по-прежнему требуют `utility_routines.h` первым.
- `avproto.h` больше не отключает три предупреждения GCC для каждого включающего его файла.
- `__util$readparams`, объявленная, но не определённая, удалена.
- `__util$movqhead(void *que, void *ent)` и `__util$movqtail(void *que, void *ent)` принимают сам элемент; они были объявлены с `void **ent` **(проверьте)**. Макросы не изменились.
- `$ILE3INI(item, code, len, ptr, retlen)` — макрос-оператор, компилирующийся с обычными аргументами; первый аргумент у него объект, а не указатель. Раскладка ILE3 (код первым) документирована.
- `__util$sub_time` и `__util$cmp_time` принимают указатели на `const`. `__util$pattern_match` называет свои параметры в заголовке.
- `__tlv$put` принимает размер PDU как `unsigned`; был `unsigned short`.
- `CLI_PQDESC.flag` документирован как используемый (отрицаемый, список, обязательный); `__cli$show_ctx` объявлена; `__cli$get_keyword` новая (раздел 8.4).

Журнал

- Доля секунды в записи — настоящие миллисекунды, от 000 до 999, на любой платформе **(проверьте)**. Она была наносекундами, сдвинутыми на 20 бит (от 0 до около 953), а в Windows — шестью цифрами.
- `$LOG`, `$LOG_FAO` и `__util$log2buf` возвращают записанную серьёзность. Бит syslog снимается до проверки серьёзности, так что помеченный вызов больше не возвращает 5 **(проверьте)**. Отладочные формы (`__util$logd`, `__util$logd_fao`) учитывают бит syslog, который прежде игнорировали.
- Приоритет syslog — `LOG_INFO` для серьёзности с установленным младшим битом и `LOG_ERR` иначе; `LOG_PID` больше не подмешивается в приоритет. (Раньше каждое сообщение уходило как `LOG_INFO`.)
- `__util$syslog` без заданного сборщика — тихий `STS$K_SUCCESS`: никаких `perror`,

`abort()` и сообщений в стандартную ошибку. Сокет создаётся один раз, потокобезопасно. `__util$deflog` игнорирует, с сообщением, адрес сборщика, не являющийся IPv4.
- Отладочная запись без имени модуля печатает `[routine:line]`, а не `[(null)\routine:line]`; `$TRACE` печатает один обратный слэш, а не два.
- `__util$faohex` принимает `outlen`, равный NULL.

FAO и время

- Сканер никогда не читает за концом управляющей строки: строка, оканчивающаяся внутри директивы (`!`, `!A`, `!%`, `!3*`), — это `FAO$K_BADDIR`.
- `!%D`, `!%T` и `__util$asctim` используют один общий отрисовщик. Отрицательное время — дельта (`dddd hh:mm:ss.cc` или `hh:mm:ss.cc`), нуль — «сейчас», год выше 9999 печатается полностью.
- `__util$bintim` отвергает абсолютную дату до 01-JAN-1970 с `FAO$K_IVTIME` **(проверьте)**. Раньше она возвращала отрицательное число, то есть дельту.
- Текст поля `!n<...!>` свыше 512 октетов обрезается с `FAO$K_FLDOVF` (раньше молча). Управляющая строка, которой нужно больше параметров, чем дано `__util$faol`, получает нули и `FAO$K_NOPRM`.
- `__util$cmp_time` возвращает знак разности (на практике -1, 0, 1) и не может переполниться; `__util$sub_time` возвращает нормализованный `timespec`. `__util$isinrange2` включает границы **(проверьте)**.

Очереди и буферы

- `__util$movqhead` и `__util$movqtail` работают; они переставляют голову, хвост и связи.
- `$REMQHEAD` и `$REMQTAIL` на пустой очереди: `STS$K_SUCCESS`, `*ent` = NULL, `*count` = 0 **(проверьте)**; `$CLRQUE` устанавливает `*count` в 0. `$REMQENT` элемента, которого нет в очереди, в том числе пустой, — `STS$K_ERROR`.
- `__util$bzero` очищает каждый октет. `__util$iszero` не читает невыровненных слов и видит каждый октет там, где `long` равен четырём байтам. `__util$fill` отвергает пустой образец (она зацикливалась навсегда).
- `__util$strelem` верна для любого индекса и оставляет `dst` пустым, когда такого куска нет.
- `__util$str2asc` обрезает на 254 октетах и держит завершающий нуль внутри `sts[]`. `__util$strcat_rx` может заполнить буфер до `dstsz - 1` и никогда не пишет в `dst[dstsz]`; `__util$collapse` больше не нуждается в октете сверх выданного; `__util$mask2sym` при проверке места учитывает разделитель и нуль.
- `__util$uncomment` пропускает части в кавычках, одинарных или двойных **(проверьте)**: `!` внутри значения в кавычках в файле настройки больше не начинает комментарий.
- `__util$pattern_match` написана заново: `*` в проверяемой строке — обычный октет (`"*ba"` соответствует `"*a"`), аргумент NULL даёт 0.
- `__util$crc32c` и `__util$faohex` документированы такими, какие они есть.

Параметры и файлы настройки

- `__util$getparams`, `__util$readconfig` и `__util$showparams` используют новые общие помощники.
- `OPTS$K_INT` сохраняет ровно 1, 2, 4 или 8 октетов, как говорит размер (0 — это `int`). Число, не помещающееся или имеющее текст после себя, — неверное значение **(проверьте)**.
- Неверное значение, неизвестный параметр, неоднозначное сокращение или лишнее слово заставляют `__util$getparams` возвращать `STS$K_ERROR`; раньше она возвращала успех **(проверьте)**. Остальные аргументы всё равно обрабатываются.
- Имя можно сократить до однозначной приставки, как в командной строке, так и в файле; полное имя побеждает. Регистр не имеет значения.
- В файле настройки параметр, которого нет в таблице, записывается в журнал как предупреждение и даёт `STS$K_INFO` (младший бит установлен); `OPTS$K_CONF` внутри файла — ошибка; строка свыше 254 октетов отбрасывается целиком.

- `OPTS$K_PORT` — от 0 до 65535 в порядке байтов узла (как документировано); `OPTS$K_STR` останавливается на 254 октетах; `argv` больше не изменяется; сообщения идут через `$LOG_FAO`.

Командная строка

- Глагол, написанный полностью, побеждает более длинный, началом которого он служит (`SET`, `SETUP`); однозначное сокращение выбирает; глаголы с одним действием, параметрами, квалификаторами и подглаголами — синонимы.
- Лишнее слово — ошибка (`Too many parameters`), в том числе для глагола без параметров. Глагол-контейнер без подглагола говорит `Too few arguments`.
- Несопоставленный `/name` принимается за путь только для ожидающего параметра `CLI$K_FILE` или `CLI$K_DEVICE`; иначе, как и для любого неизвестного `--name`, это ошибка `Unrecognized qualifier` **(проверьте)**. Раньше он становился следующим необязательным параметром.
- `/q value` (без `=`) получает сообщение, называющее способ исправления.
- `CLI$K_KWD` сохраняет полное ключевое слово таблицы; `__cli$get_keyword` возвращает его запись таблицы. Программу, сравнивавшую сохранённый текст с набранным сокращением, необходимо изменить **(проверьте)**.
- `CLI$K_NUM` принимает только беззнаковое число, которое помещается; `CLI$K_DATE` проверяет календарь и ничего не принимает после времени.
- `__cli$get_value` для отсутствующего элемента — тихий `STS$K_ERROR`, в том числе с `CLI$M_OPSIGNAL`; идиома сброса `opts` перед диспетчеризацией не нужна. Присутствует с пустым значением: `STS$K_WARN`.
- `/HELP` понимается для подглагола. `__cli$show_verbs` перечисляет таблицу подглаголов один раз и оставляет уровень таким, каким нашла.

Рутины TLV

- `__tlv$put` проверяет свободное место по реальной длине записи, отвергает id тега выше 4095 и тип выше 15 и завершается неудачей (ничего не записано) там, где прежде переполняла. Запись UUID добавляет к PDU свою реальную длину. `valsz` для слова, длинного слова, квадрослова или UUID должен быть не меньше размера типа; **нуль отвергается (проверьте)**.
- `__tlv$hset` отвергает сигнатуру свыше восьми октетов (раньше обрезала) и NULL. `__tlv$hget` не записывает ни одного из своих выходов при несовпадении (раньше сохраняла `STS$K_ERROR` в `*csr`).
- `__tlv$get` проверяет длину в линии записи фиксированного размера и сообщает об усечённой как об ошибке; `__tlv$get` и `__tlv$lookup` сообщают о записи, выходящей за PDU. `__tlv$dump` печатает значение кусками по 64 октета и больше не переполняет свой буфер.

Тесты и документы

- Семь тестовых программ новые: `fao_edge`, `tlv_hostile`, `cli_rules`, `queue`, `strings`, `params`, `log_misc`. `test_pattern_match` сравнивает с эталоном на динамическом программировании. `tests/run.sh` запускает все.
- Строка об авторстве и инициалы в историях исправлены. Шапки `__util$movc5` и сопоставителя шаблонов написаны заново. Заголовки модулей в исходных текстах сокращены до необходимого.